BASIC CONCEPTS

- **Overview**
  - Programming Paradigms
  - Values, Sets, Arrays, Templates
  - Indexer, Lambda Expressions, Iterators, and Recursion
- **References**

  - Richard F. Gilberg and Behrouz A. Forouzan: Data Structures - A Pseudocode Approach with C. 2nd Edition. Thomson (2005)
  - Russ Miller and Laurence Boxer: Algorithms Sequential & Parallel. 2nd Edition. Charles River Media Inc. (2005)
  - Stanley B. Lippman, Josée Lajoie, and Barbara E. Moo: C++ Primer. 5th Edition. Addison-Wesley (2013)
  - Scott Meyers: Effective Modern C++. O'Reilly (2014)
  - Anthony Williams: C++ Concurrency in Action - Practical Multithreading. Manning Publications Co. (2012)
  - Marius Bancila: Template Metaprogramming with C++. Packt Publishing (2022)

PROGRAMMING PARADIGMS

- **Imperative style:**

  program = algorithms + data

- **Functional style**:

  program = function • function

- **Logic programming style**:

  program = facts + rules

- **Object-oriented style**:

  program = objects + messages

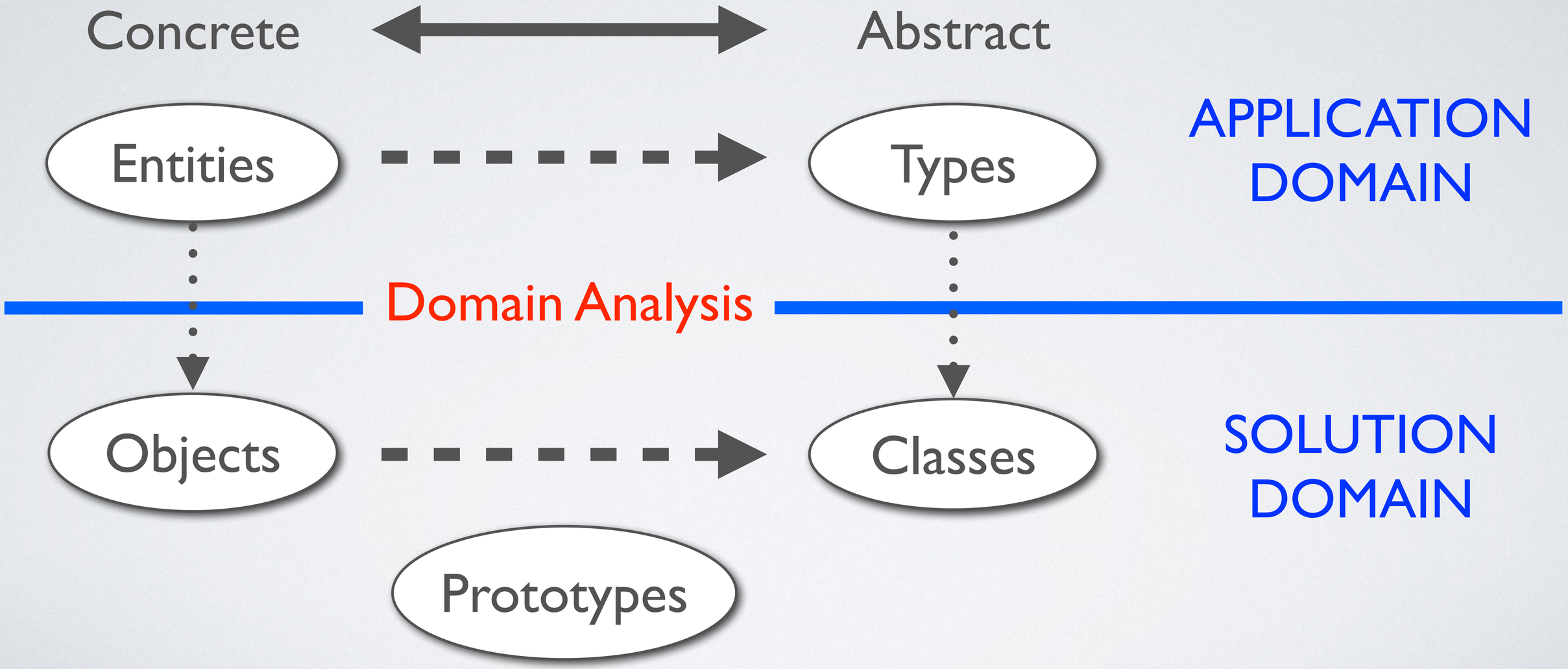
- Other styles and paradigms:

  blackboard, events, pipes and filters, constraints, lists, …

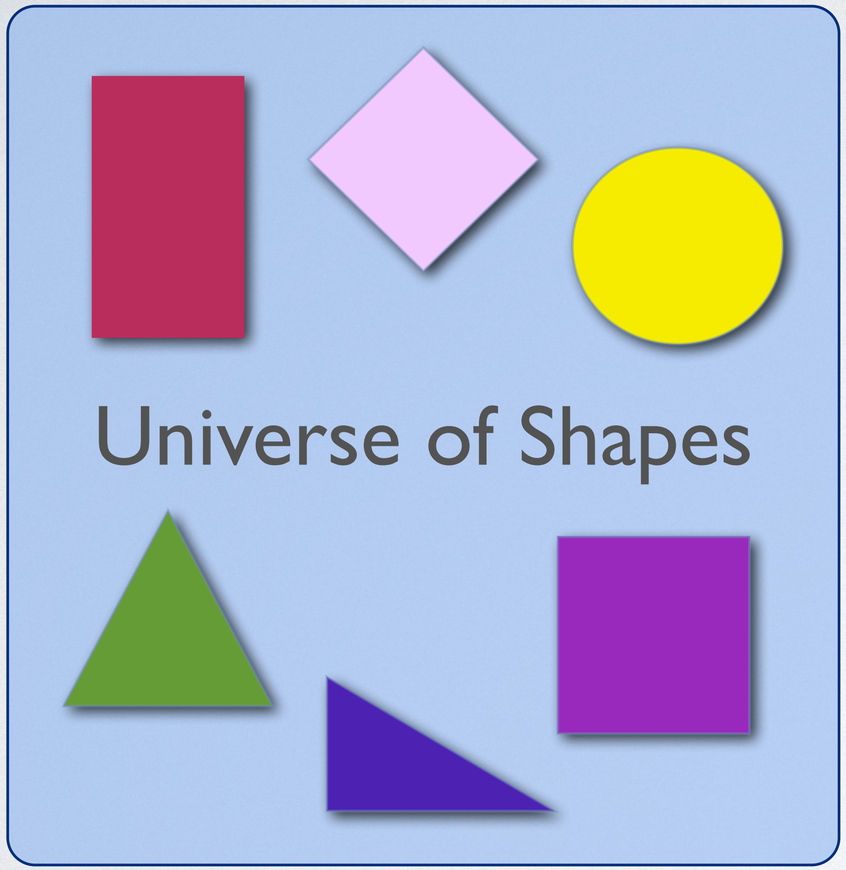
OBJECT-ORIENTED SOFTWARE DEVELOPMENT

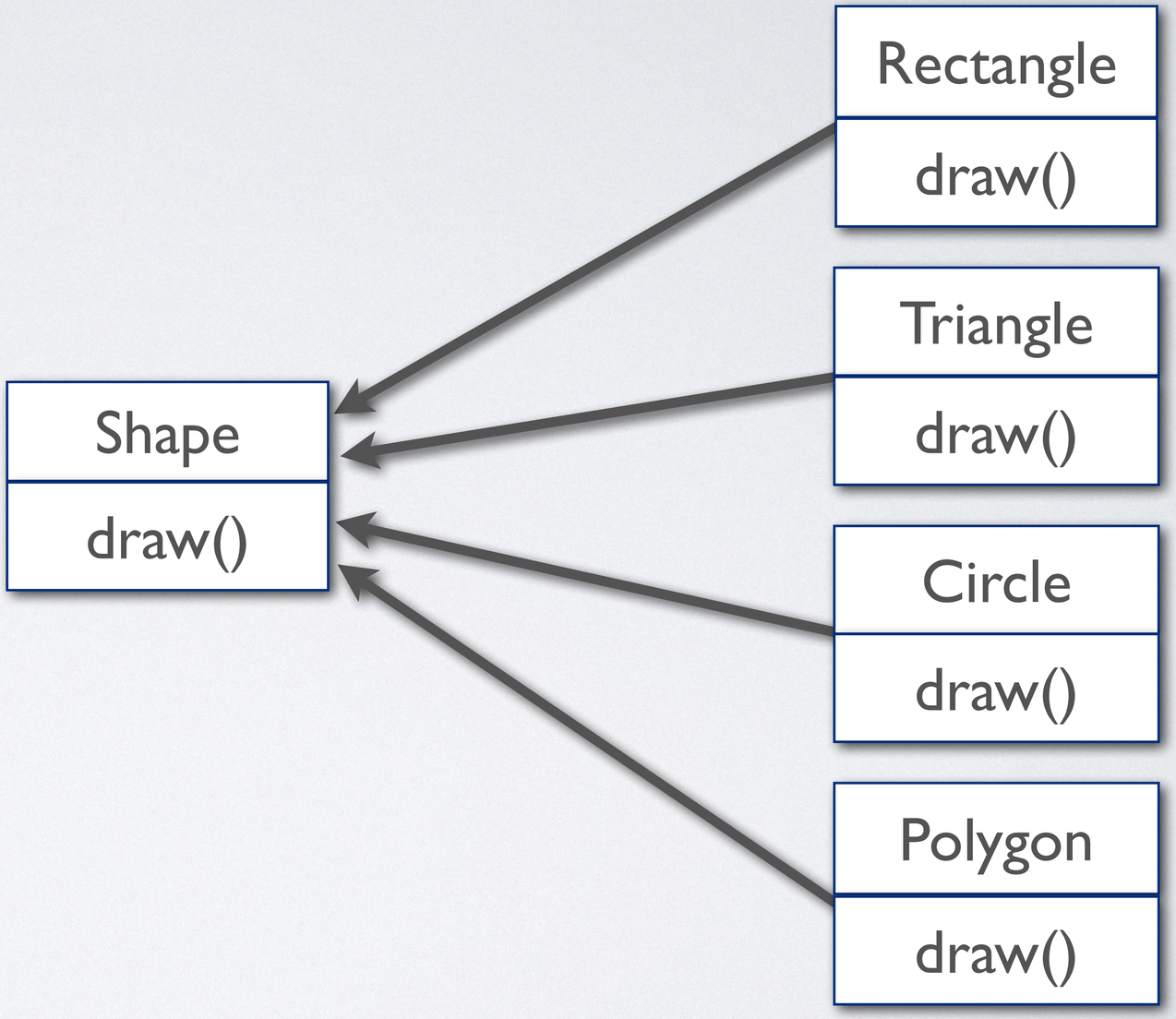
- Object-oriented programming is about
  - Object-oriented software development
  - Using an object-oriented programming language
- Object-oriented software development is
  - An evolutionary step refining earlier techniques
  - A revolutionary idea perfecting earlier methods

OBJECT-ORIENTED DESIGN



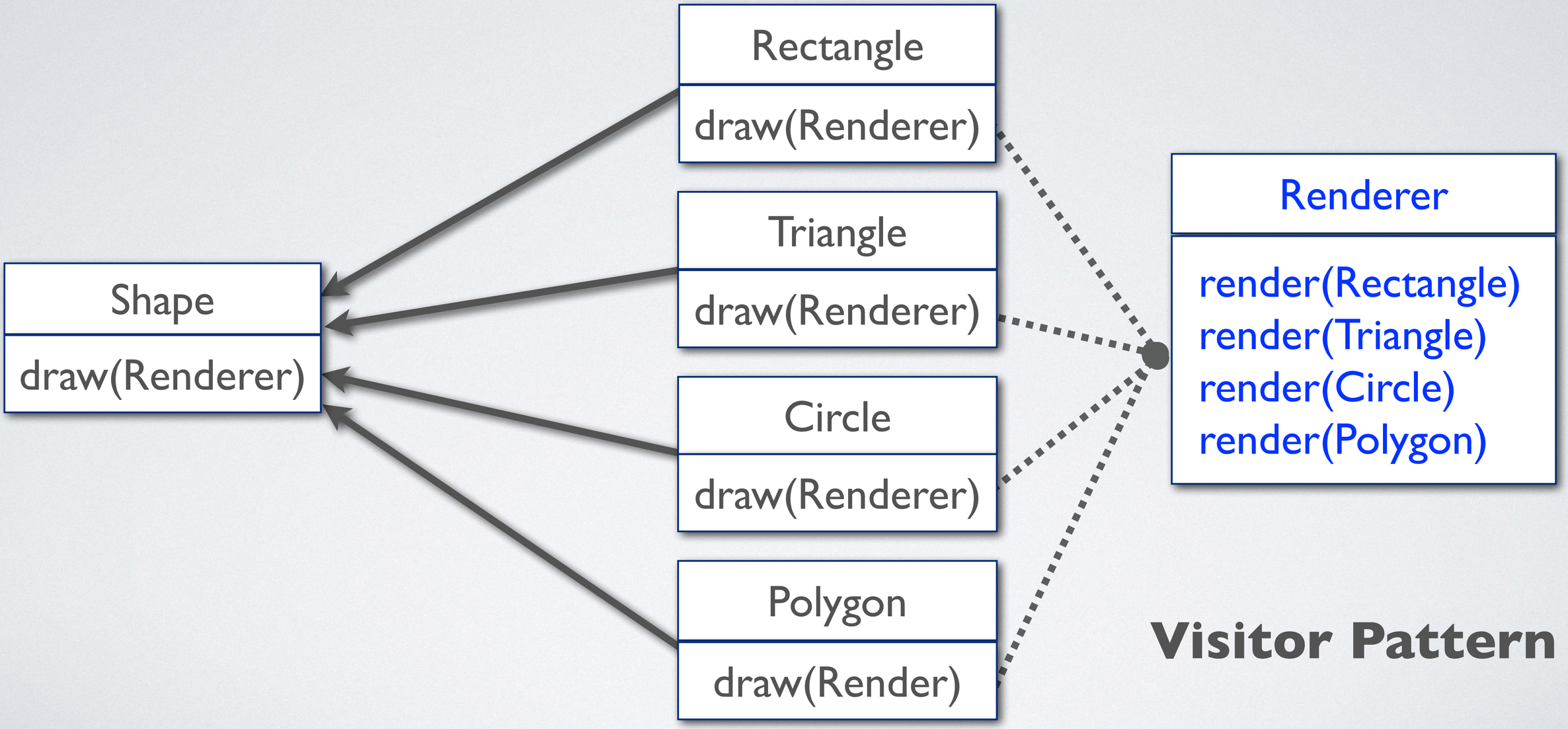
CONCRETE VS ABSTRACT





NOT SO FAST

SHAPES CAN'T DRAW THEMSELVES



Visitor Pattern

WHY IS OBJECT-ORIENTED SOFTWARE DEVELOPMENT POPULAR?

- The object-oriented development approach
  - Naturally captures real life
  - Scales well from trivial to complex tasks
  - Focuses on responsibilities, reuse, and composition

VALUES

- In computer science, we classify as a value everything that may be evaluated, stored, incorporated into a data structure, passed as an argument to a procedure or function, returned as a function result, and so on.
- We use "expressions" to denote values, as in mathematics.
- Which values are supported by a specific programming environment heavily depends on the underlying paradigm and its application domain.
- Most programming environments support basic value sets like truth values, integers, real numbers, records, lists, etc.

CONSTANTS

- Constants are named abstractions of values.
- Constants are used to assign a user-defined meaning to a value.
- Examples:
  - **EOF** = -1
  - **TRUE** = 1
  - **FALSE** = 0
  - **PI** = 3.1415927
  - **MESSAGE** = "Welcome to DSP"
- Constants do not have an address or defined (programmatically usable) location.
- At compile time, applications of constants are substituted by their corresponding definition.

PRIMITIVE VALUES

- Primitive values are values whose representation cannot be further decomposed. Some of these values are implementation and platform-dependent.
- Examples:
  - Truth values,
  - Integers,
  - Characters,
  - Strings,
  - Enumerators,
  - Real numbers.

-1

"Hello World!"

3.14159

false

Red

COMPOSITE VALUES

- Composite values are composed of primitive values and other composite values. The layout of composite values is, in general, implementation-dependent.

- Examples:
  - Records
  - Arrays
  - Enumerations
  - Sets
  - Lists
  - Tuples
  - Files

```
struct Point
{
    int x;
    int y;
} gPoint = { 1, 2 };

enum class Color
{
    Red,
    Blue,
    Green
};
```

POINTERS

- Pointers denote locations (or the address) of other values in memory.
- Most programming languages use pointers to access objects stored in memory. C++, in addition, supports operations to obtain and manipulate pointers.
- In C++, we can obtain the address of any lvalue and function. The latter is called a pointer to a function, which, before lambda expressions, is the only way to pass a (statically defined) function as argument to another function.

```
int i = 10;
int* ptr_to_i = &i;              // using & to obtain address of i

int convert( const char* aCString,  int(*aMap)(char aChar))
{
   assert( aCString != nullptr );

   int Result = 0;

   while ( *aCString ) // using * to deference pointer aCString
   {
      Result = Result * 10 + aMap( *aCString );
      aCString++;   // pointer arithmetic, advance to next character
   }

   return Result;
}

int ctoi( char aChar )
{
   assert( isdigit( aChar ) );    // debug-mode assertion testing

   return aChar - '0';            // characters are integers, '0' == 0x30
}

…

const char* IString = "12345";

std::cout << "\"12345\": " << convert( IString, ctoi ) << std::endl;
```

SETS

- A set is a collection of elements (or values), possibly empty.
- All elements satisfy a possibly complex characterizing property. Formally, we write:

$$\mathbf{\{ x \mid P(x) = True \}}$$

- to define a set, where all elements satisfy the property **P(x)**.
- The basic axiom of set theory is that there exists an empty set ∅ with no elements. Formally,

$$\mathbf{\forall x,\ x \notin \emptyset}$$

In words, "for every x, x is not an element of ∅."

INDUCTIVE SPECIFICATION

- Sometimes, to define a set explicitly is difficult if the elements of the set have a complex structure.
- However, it may be easy to define the set in terms of itself. This process is called inductive specification or recursive specification. Inductive specification yields a finite recipe for constructing a possibly infinite set of values.
- Example:

  Let set S be the smallest set of natural numbers divisible by 3, satisfying the following two properties:

  - $0 \in S$, and
  - Whenever $x \in S$, then $x + 3 \in S$.

  The first property is called base clause, and the second is called inductive/recursive clause. An inductive specification may have multiple base and inductive clauses.

THE "SMALLEST SET"

- If we use inductive specification, we always define the smallest set that satisfies all given properties. That is, inductive specification is free of redundancy.

- It is easy to see that there can be only one such set:

  If S_1 and S_2 satisfy all given properties and both are the smallest, then we have $S_1 \subseteq S_2$ (since S_1 is the smallest), and $S_2 \subseteq S_1$ (since S_2 is the smallest), hence $S_1 = S_2$. Q.E.D.

INDEXED SETS

- Sets are unordered collections of data elements.

- To obtain an ordering relation over the elements of a given set, we can assign each element in that set a unique value of another, totally ordered, set **I** (**I** is irreflexive, antisymmetric, transitive, and connected):

$$S_{\mathbf{I}} = \{ a_i \mid a \in S, i \in \mathbf{I} \}$$

$S_{\mathbf{I}}$ is called the "indexed set" of S.

SOME INDEXED SETS

- Let $A = \{ a, b, c, d \}$ and $\mathbf{I} = \{ 1, 2, 3, 4 \} \subset \mathcal{N}$, then

$$A_{\mathbf{I}} = \{ a_1, b_2, c_3, d_4 \}$$

- Let $A = \{ a, b, c, d \}$ and $\mathbf{I} = \{ \text{“1”}, \text{“2”}, \text{“3”}, \text{“4”} \} \subset (\mathbf{S} \times \mathbf{S}, <)$, then

$$A_{\mathbf{I}} = \{ a_{\text{“1”}}, b_{\text{“2”}}, c_{\text{“3”}}, d_{\text{“4”}} \}$$

- Arrays are indexed sets.

PAIRS AND MAPS

- Let A and B be sets. The Cartesian product of A and B, denoted by $A \times B$, is the set of all ordered pairs (a, b) where $a \in A$ and $b \in B$:

$$A \times B = \{ (a,b) \mid a \in A \text{ and } b \in B \}$$

- A map is an associative container whose elements are defined as key-value pairs. The key serves as an index into the map, and the value represents the data being stored and retrieved.

ASSOCIATIVE ARRAYS (DICTIONARIES)

- An associative array is a map in which elements are indexed by a key rather than their position. Technically, it is a partial function whose domain is a set of keys, and its codomain is a set of values mapped by keys.

- We use the standard array indexing syntax to denote a value lookup. If the associative array does not define a corresponding mapping, then an exception has to occur:

$$a[i] = \begin{cases} v & \text{if } i \mapsto v \text{ in } a \\ \bot & \text{otherwise} \end{cases}$$

- Example:

  a = { (“u” ↦345), (“v” ↦ 2), (“w” ↦ 39), (“x” ↦ 5) }

  a[“w”] = 39

  a[“z”] = ⊥

AN INDEXER AS CLASS ADAPTER

- We can define an indexer as a class adapter:

```
#include <cstddef>              // definition of size_t
#include <string>               // definition of std::string

class SimpleAssociativeArray
{
public:
    using value_type = int;                     // associated value type of SimpleAssociativeArray
    using difference_type = std::ptrdiff_t;     // associated pointer difference type of SimpleAssociativeArray

    SimpleAssociativeArray( int aArray[], size_t aSize ) noexcept;

    size_t size() const noexcept;

    int& operator[]( size_t aIndex ) const;
    int& operator[]( const std::string& aKey ) const;

private:
    int* fArray;            // non-owning pointer to data array
    size_t fSize;
};
```

TYPE ALIAS DECLARATION

```
using alias = type;
```

- A type alias is a name that refers to a previously defined type.
- Type aliases are commonly used for three purposes:
  - To hide the implementation of a given type.
  - To streamline complex type definitions, making them easier to understand, and
  - To allow a single type to be used in different contexts under different names.
- Type aliases establish a nominal equivalence between types.
- Type aliases are similar to **typedef**. However, type aliases are better suited when creating alias templates.

CONSTRUCTOR

```
SimpleAssociativeArray::SimpleAssociativeArray( int aArray[], size_t aSize ) noexcept :
    fArray(aArray),
    fSize(aSize)
{}
```

- When we pass an array to a function, the array type "decays" to a pointer to the first element in C++.
- Consequently, we also have to pass the size of the array.
- The member variable fArray must be pointer to **int** as the type of parameter aArray decays to a pointer. Pointer variables can be used like arrays using the selection **operator[]**.

NESTED TYPES (MEMBER TYPES)

```
using value_type = int;                  // associated value type of SimpleAssociativeArray
using difference_type = std::ptrdiff_t;  // associated pointer difference type of SimpleAssociativeArray
```

- We can declare and define types within a class definition.
- Nested types obey the access level modifiers (i.e. **private**, **protected**, and **public**).
- Nested types describe the types being used by the host data type. The programmer does not have to figure them out manually. Users of a data type can refer to the nested types instead of using, possibly dangerous, type casts.
- Within the scope of SimpleAssociativeArray, type name value_type refers to the value type (i.e. **int**), and type name difference_type refers to the signed integer type of the result of subtracting two pointers associated with type SimpleAssociativeArray.
- Member types describe the types used in a data type and play an important role in traits and concepts.

SIZE

```
size_t SimpleAssociativeArray::size() const noexcept
{
    return fSize;
}
```

- The function size() returns the size of the wrapped array.

INDEXED SET ACCESS - ARRAY SEMANTICS

- To define standard array semantics, we overload the index **operator[]**.
- The index is of integral type and denotes the position in the array.
- Integral types define a total ordering relation. Hence, every element has a unique position in the array.

```
int& SimpleAssociativeArray::operator[]( size_t aIndex ) const
{
    assert( aIndex < fSize );        // debug range check

    return fArray[aIndex];
}
```

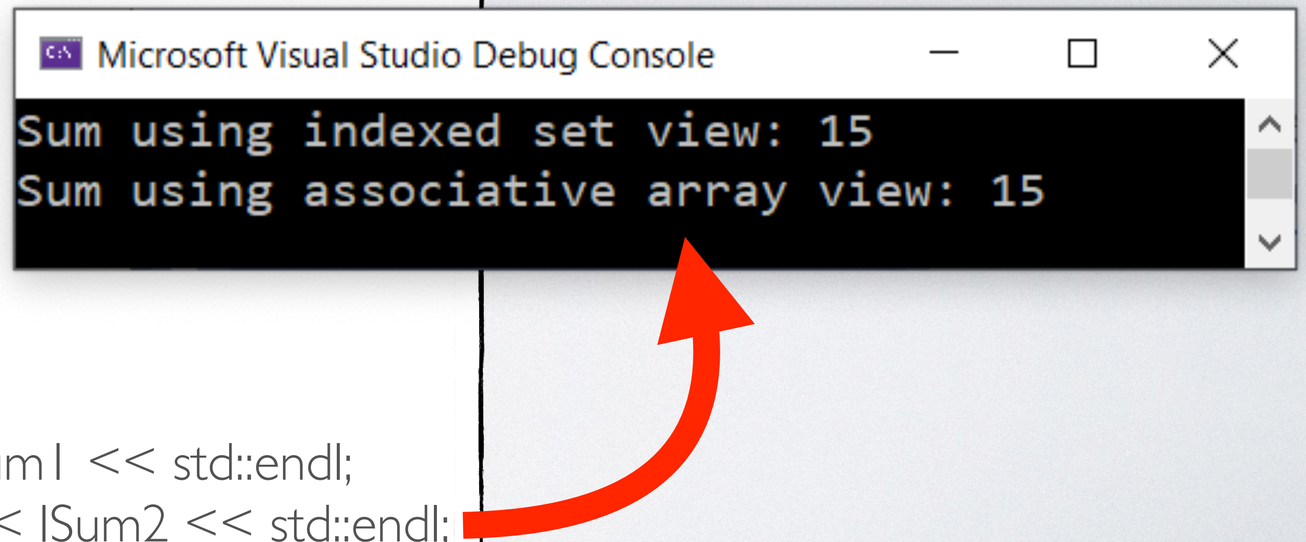
INDEXED SET ACCESS - ASSOCIATIVE ARRAY SEMANTICS

```
int& SimpleAssociativeArray::operator[]( const std::string& aKey ) const
{
    size_t lIndex = 0;

    for ( char c : aKey )           // scan key string
    {
        if ( std::isdigit( c ) )
        {
          // convert string to number
          lIndex = lIndex * 10 + static_cast<size_t>(c) - '0';
        }
        else
        {
          break;
        }
    }

    return (*this)[lIndex];         // forward to operator[]( size_t aIndex )
}
```

- We can define a map from strings to numbers.
- The **for**-loop works similarly to the function std::stoi().
- We forward the obtained index to the integer-based index operator, which performs range checking and yields the element.

APPLY OPERATIONS

```
int IArray[] = { 1, 2, 3, 4, 5 };
std::string IKeys[] = { "0", "1", "2", "3", "4" };
constexpr int ISize = sizeof( IArray ) / sizeof( int );

SimpleAssociativeArray IDict( IArray, ISize );

int ISum1 = 0;
int ISum2 = 0;

for ( size_t i = 0; i < ISize; i++ )
{
    ISum1 += IDict[i];
    ISum2 += IDict[IKeys[i]];
}

std::cout << "Sum using indexed set view: " << ISum1 << std::endl;
std::cout << "Sum using associative array view: " << ISum2 << std::endl;
```



TEMPLATES

TEMPLATES: DEFINING GENERIC ABSTRACTIONS IN C++

- Templates are the basis for generic programming in C++.
- C++ requires all variables to have a specific type, explicitly declared or deduced by the compiler.
- However, many data structures and algorithms are type-agnostic and work the same irrespective of the values they operate on.
- Templates enable us to define the operations of classes and function as blueprints and let the user specify/instantiate what concrete types those operations should work on. The latter are called specializations.
- **Note**, every time a given template is instantiated with type parameters that have not been used before, a new version of the class or function is generated.

DEFINING A TEMPLATE

- A template is a parameterized abstraction over a function, class, variable, or type.
- From the language-theoretical perspective, templates are 2nd-order functions, from types and objects to functions, classes, variables, or types.
- To instantiate a class template, we supply the desired types as actual template parameters so that the C++ compiler can synthesize a specialized class for the template.
- Further information:
  - Nicolai M. Josuttis: C++17 - The Complete Guide. https://leanpub.com/cpp17 (2019)
  - Marius Bancila: Template Metaprogramming with C++. Packt Publishing (2022)

FUNCTION TEMPLATES

Placeholder for a type.

```
template<typename T1, ..., typename Tn>
T1 AFunction( T2 aParam1, ..., Tn aParamn )
{
    // body of function
};

type1 value = AFunction<type1, ..., typen>( value1, ..., valuen );
```

$T_1, \ldots, T_n$ occur in function signature.

$T_1, \ldots, T_n$ instantiated in function specialization.

Types of $value_1, \ldots, value_n$ must match $type_1, \ldots, type_n$

INSERT SORT FUNCTION TEMPLATE

T is element type.

The ordering criterion can be changed programmatically.

Parameter with default type.

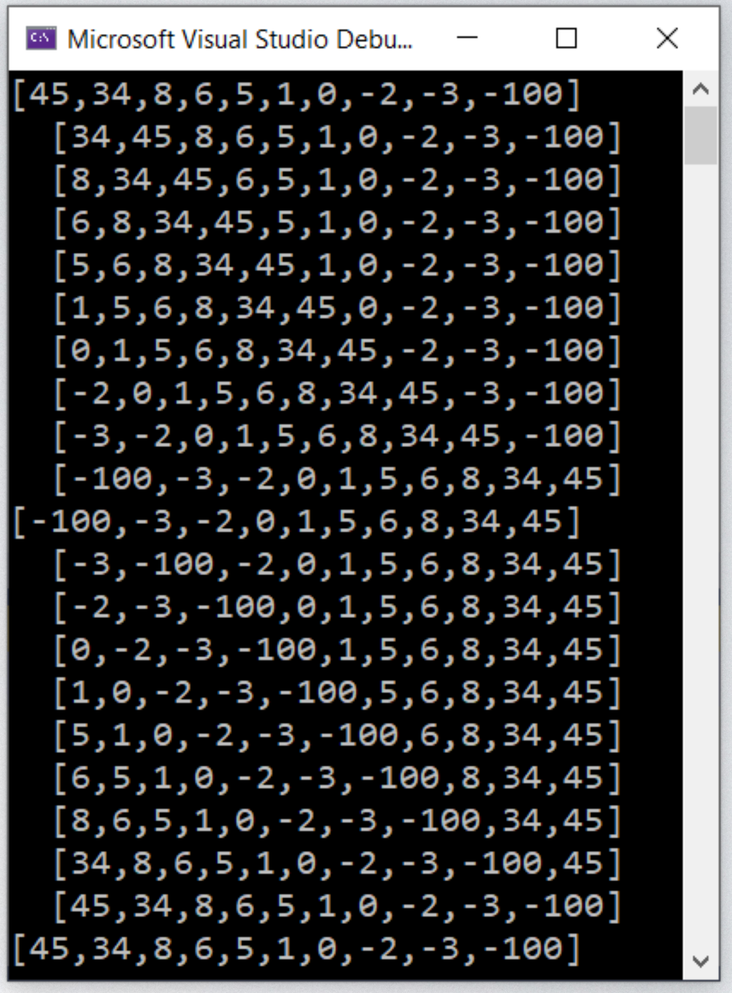
std::swap is also a template function

```
template<typename T, typename Op = std::greater<T> >
void performInsertionSort( std::vector<T>& aArray, Op aTest = Op{},
                           std::ostream& aOStream = std::cout )
{
    size_t i = 1;

    while ( i < aArray.size() )
    {
        size_t j = i++;

        while ( j > 0 && aTest( aArray[j-1], aArray[j] ) )
        {
            std::swap( aArray[j-1], aArray[j] );

            j--;
        }

        sendToStream( aOStream, aArray, 2u ) << std::endl;
    }
}
```

SORT AN ARRAY

The compiler can deduce instantiation types.

```
std::vector IArrayA { 45, 34, 8, 6, 5, 1, 0, -2, -3, -100 };

sendToStream( std::cout, IArrayA ) << std::endl;
performInsertionSort( IArrayA );
sendToStream( std::cout, IArrayA ) << std::endl;
sendToStream( std::cout, IArrayA ) << std::endl;
performInsertionSort( IArrayA, std::less<int>{} );
sendToStream( std::cout, IArrayA ) << std::endl;
```



CLASS TEMPLATES

```
template<typename T1, ..., typename Tn>
class AClassTemplate
{
    // class specification
};

AClassTemplate<type1, …, typen> obj{};
```

Placeholder for a type.

$T_1, \ldots, T_n$ occur in class specification.

$T_1, \ldots, T_n$ instantiated in class specialization.

new initializer syntax

INSERTION SORTER CLASS TEMPLATE

```
template<typename T>
class InsertionSorter
{
public:

    using array_type = std::vector<T>;

    InsertionSorter( const array_type& aData ) : fData(aData) {}

    template<typename Op = std::greater<T>>
    void operator()( Op aTest = Op{}, std::ostream& aOStream = std::cout )
    {
        performInsertionSort( fData, aTest, aOStream );
    }

    const array_type& data() const { return fData; }

private:

    array_type fData;
};
```

T is element type.

The ordering criterion is a parameter of the sorting process, not the class itself.

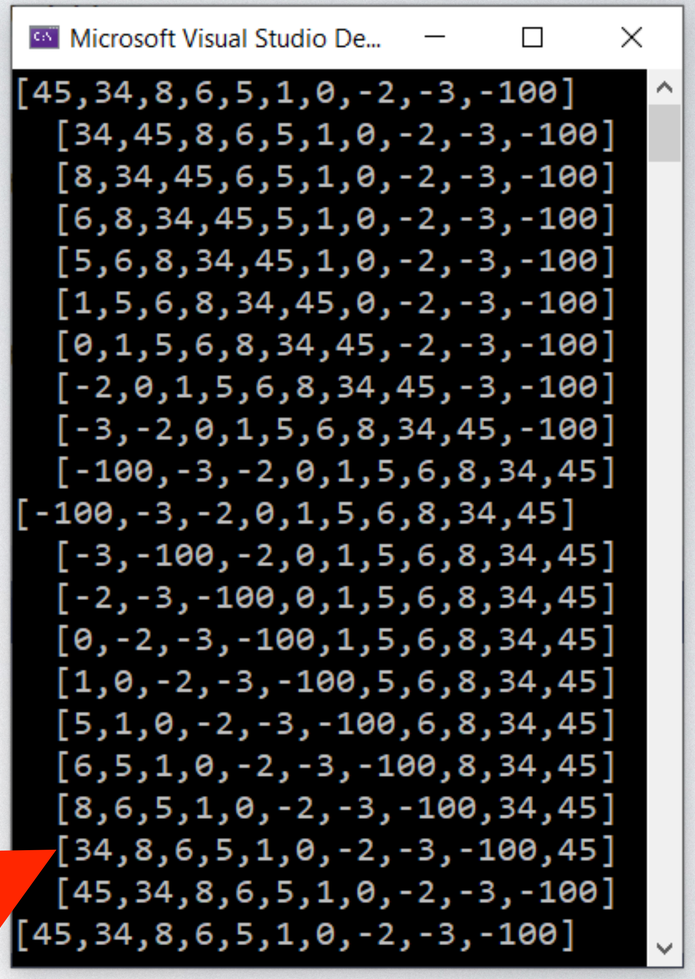
Parameter with default type.

Sorter private vector

SORT A VECTOR

The compiler deduces all instantiation types.

```
std::vector IArrayB { 45, 34, 8, 6, 5, 1, 0, -2, -3, -100 };
InsertionSorter ISorter( IArrayB );

sendToStream( std::cout, ISorter.data() ) << std::endl;
ISorter();
sendToStream( std::cout, ISorter.data() ) << std::endl;
sendToStream( std::cout, ISorter.data() ) << std::endl;
ISorter( std::less<int>{} );
sendToStream( std::cout, ISorter.data() ) << std::endl;
```



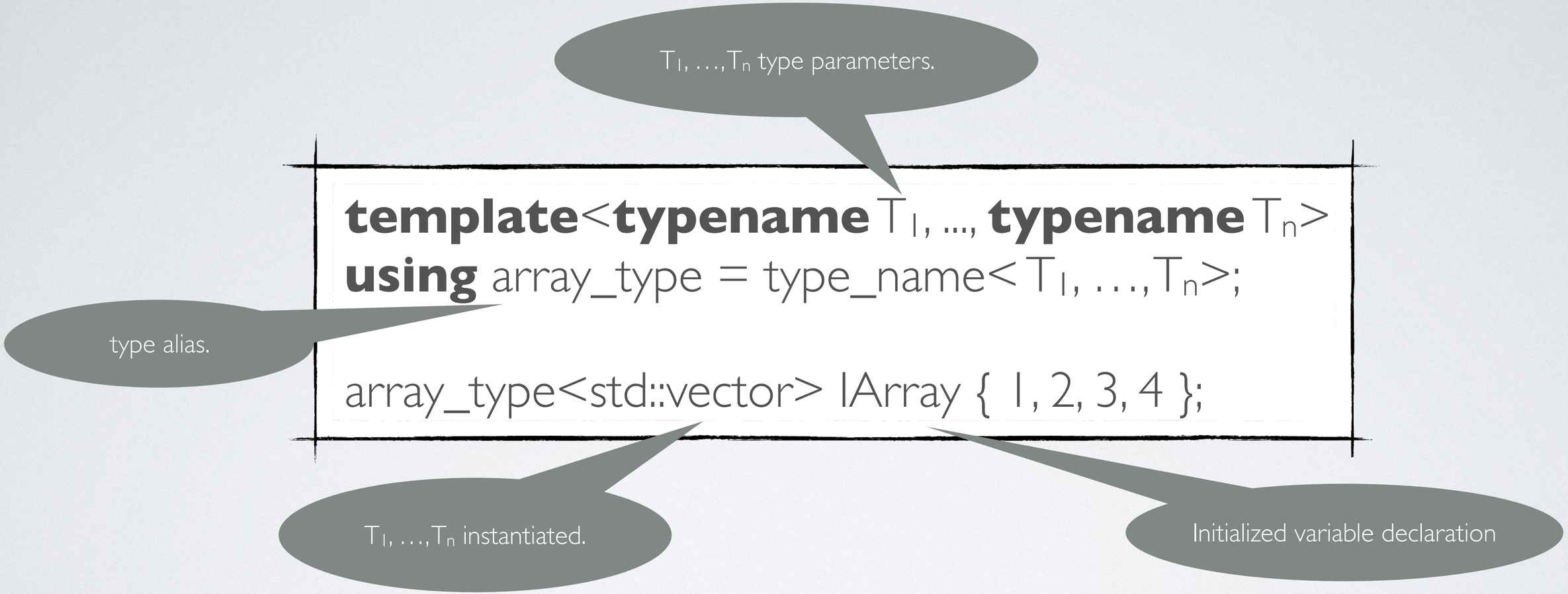
VARIABLE TEMPLATES

Placeholder for a type.

```
template<typename T>
T pi_v = T(3.14159265358979323846);

float pif = pi_v<float>;
```

Template variable pi_v instantiated to float.

ALIAS TEMPLATES



TYPE DEDUCTION

- Before C++17, we had to specify all template parameter types for all class templates. For example, we had to write:

```
std::vector<int> IData{ 45, 34, 8, 6, 5, 1, 0, -2, -3, -100 };
```

- This constraint has been relaxed in C++17. By using class template argument deduction, we can skip defining template arguments explicitly if the constructor can deduce all template parameters:

- **Note**: Template parameters have to be unambiguously deducible.

NOTE OF TEMPLATE PARAMETER DEDUCTION

The rules for template parameter deduction are rather complex. We also must consider default parameters (e.g., **typename** C = std::greater<T>) that allow us to skip explicit argument specification.

In addition, C++ allows for the specification of deduction guides to assist the compiler. (This feature is beyond the scope of COS30008.)

TEMPLATE IMPLEMENTATION

- When using templates, the C++ compiler must have access to the specification and all method implementations to instantiate the template.
- Do not confuse templates with library classes. Templates are blueprints that have to be instantiated for each distinct type of application.
- Think of templates as special forms of macros.
- When defining a template class, we need to implement everything within the class definition, usually in a header file. Templates do not require a source file (e.g., .cpp).

INDEXER

ALLOW ANY DATA TYPE TO BE INDEXED LIKE AN ARRAY

- To use a data type like an array, it must provide an overridden index **operator[]**.
- Overloading **operator[]** may be useful
  - to check for index out-of-bounds or
  - to allow for selectors other than integers.
- **Operator[]** must return an lvalue reference so that the indexed value can be used as lvalue.
- We can enforce read-only semantics by decorating the returned lvalue reference as const.

A BASIC INDEXER

trivially copyable

We can use non type parameters, e.g., N = 20.

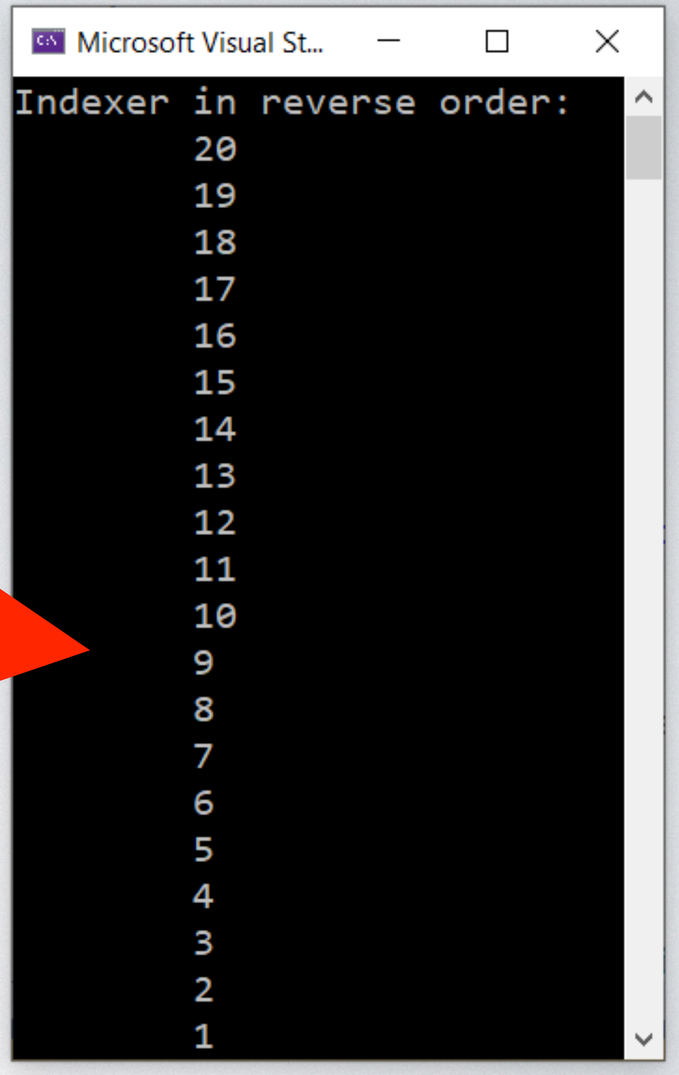
read-write lvalue reference

Debug mode test

```
template<typename T, size_t N = 20>
class BasicIndexer
{
private:
   T fElements[N];

public:

   using value_type = T;
   using difference_type = std::ptrdiff_t;

   BasicIndexer() noexcept : fElements() {}

   const size_t size() const noexcept { return N; }

   T& operator[]( size_t aIndex )
   {
      assert( aIndex < N );

      return fElements[aIndex];
   }
};
```

POPULATE BASIC INDEXER

```
using IntBasicIndexer = BasicIndexer<int>;

// Compile-time check of indexer properties using C++20 concepts and type traits.
static_assert( std::copyable<IntBasicIndexer> &&
               std::is_same_v<std::void_t<IntBasicIndexer::value_type,
                                          IntBasicIndexer::difference_type>,
                              void> );

IntBasicIndexer IIndexer;

for ( size_t i = 0; i < IIndexer.size(); i++ )
{
    IIndexer[i] = static_cast<int>(i) + 1;
}

std::cout << "Indexer in reverse order:" << std::endl;

for ( size_t i = IIndexer.size(); i > 0; )
{
    std::cout << '\t'<< IIndexer[--i] << std::endl;
}
```



TYPE TRAITS

- Type traits are a meta-programming technique that enables us to inspect properties or perform transformations of types at compile-time.
- Type traits are templates, and you can see them as meta-types.
- For example, std::void_t is a type trait (or utility meta-function) to detect ill-formed types. It yields a type alias for **void** only if all type arguments are well-formed.
- Similarly, std::is_same_v is a type trait that returns a compile-time *Boolean* value to indicate whether the two given types are the same.

CONCEPTS

- The C++20 standard adds constraints and concepts to the language that facilitate template meta-programming.
- A constraint is a modern C++ way to define requirements on template parameters. A concept is a set of named constraints.
- Concepts provide several benefits, mainly improved readability of code, better diagnostics, and reduced compilation times.
- For example, std::copyable<T> is a C++20 concept that specifies that T is an object type and supports copy and move operations. (Note, objects of BasicIndexer are trivially copyable).

USING STRINGS AS INDICES

```
#include "BasicIndexer.h"

#include <string>

template<typename T, size_t N = 20>
class IndexerByString : public BasicIndexer<T, N>
{
public:
    T& operator[]( const std::string& aKey )
    {
        size_t lIndex = 0;

        for ( size_t i = 0; i < aKey.size(); i++ )
        {
            lIndex = lIndex * 10 + (static_cast<size_t>(aKey[i]) - '0');
        }

        // Call base class operator[] (performs range check)
        return BasicIndexer<T, N>::operator[](lIndex);
    }
};
```

inheritance

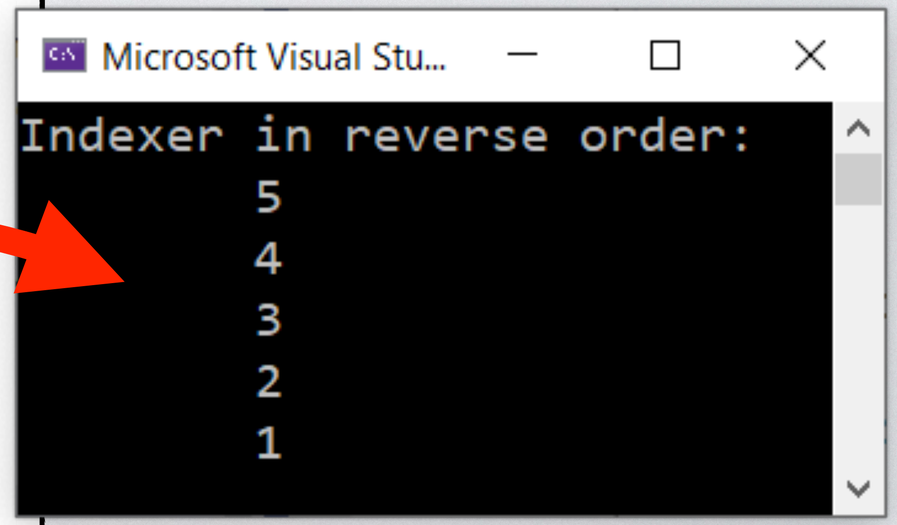
trivially copyable
delegate instance

stoi

forward request

POPULATE BASIC INDEXER

```
using IntIndexerByString = IndexerByString<int, 5>;

// Compile-time check of indexer properties using C++20 concepts and type traits.
static_assert( std::copyable<IntIndexerByString> &&
                    std::is_same_v<std::void_t<IntIndexerByString::value_type,
                                               IntIndexerByString::difference_type>,
                                   void> );

IntIndexerByString IIndexerS;
std::string IKeys[] = { "0", "1", "2", "3", "4" };

for ( size_t i = 0; i < IIndexerS.size(); i++ )
{
    IIndexerS[IKeys[i]] = static_cast<int>(i) + 1;
}

std::cout << "Indexer in reverse order:" << std::endl;

for ( size_t i = IIndexerS.size(); i > 0; )
{
    std::cout << '\t' << IIndexerS[IKeys[--i]] << std::endl;
}
```



ADDITIONAL FLEXIBILITY: LAMBDA EXPRESSIONS

HARD-CODED CONVERSION

```
T& operator[]( const std::string& aKey )
{
    size_t lIndex = 0;

    for ( size_t i = 0; i < aKey.size(); i++ )
    {
        lIndex = lIndex * 10 + (static_cast<size_t>(aKey[i]) - '0');
    }

    // Call base class operator[] (performs range check)
    return BasicIndexer<T, N>::operator[](lIndex);
}
```

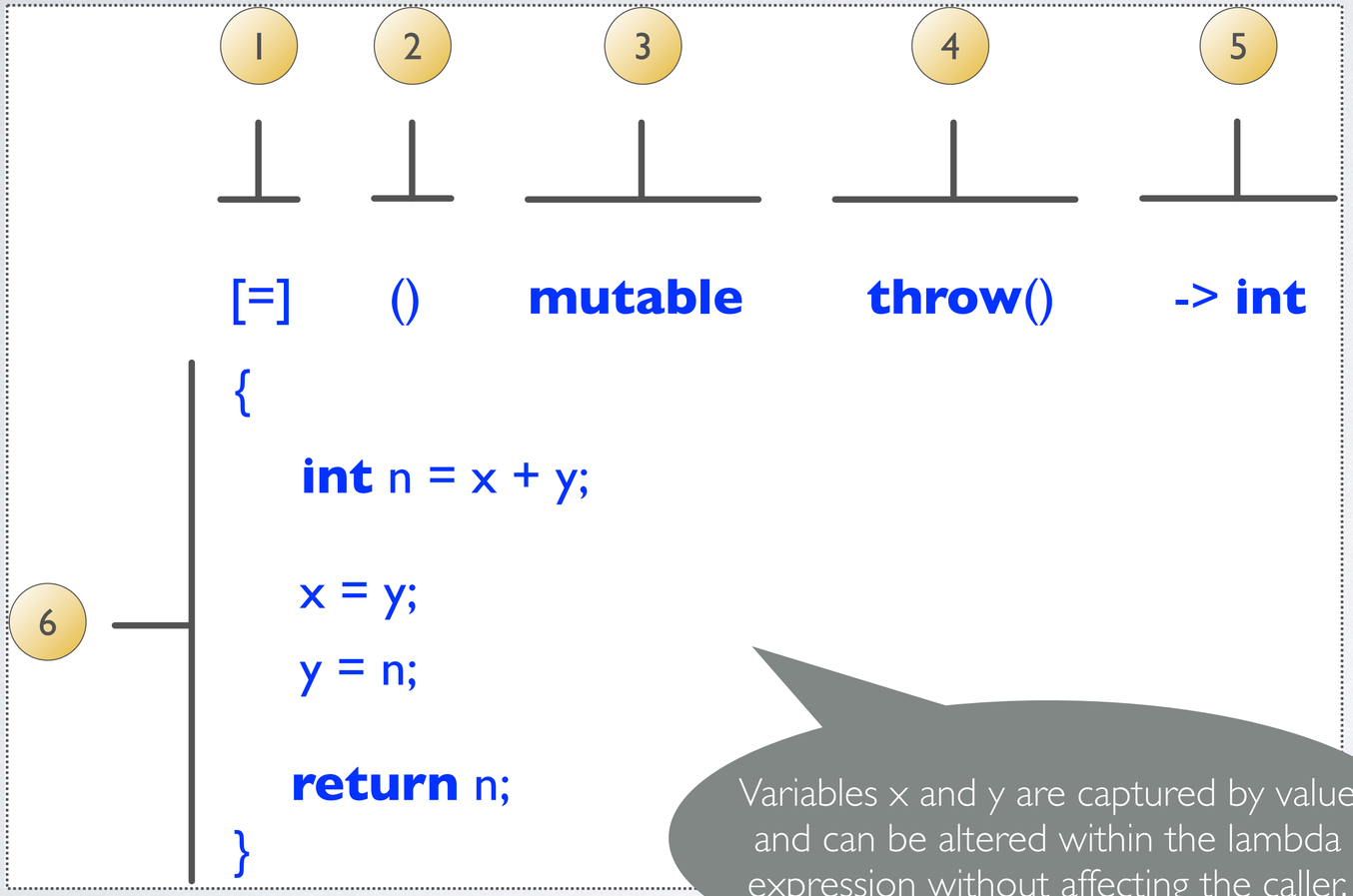
hard-coded stoi

- The string-based indexer uses a hard-coded conversion from string to size_t.
- This limits the application of the indexer.
- A better option is to define an explicit conversion function and pass it to the indexer when needed.

C++ LAMBDA EXPRESSIONS

- Since C++11, lambda expressions have been available in C++.
- A lambda expression, or just lambda, is an anonymous function object (closure) defining a callable unit of code.
- Like any function, a lambda has a return type, a parameter list, and a function body.
- Unlike functions, lambdas can be defined inside a function.
- Note, C++ supports two kinds of callable objects: objects of classes that override the call operator (i.e., **operator()**) and lambda expressions.

C++ LAMBDA (C++11 STANDARD)



Variables x and y are captured by value and can be altered within the lambda expression without affecting the caller.

1. Capture clause
2. Parameter list, optional
3. Mutable specification, optional
4. Exception specification, optional
5. Trailing return type, optional
6. Lambda body

C++ LAMBDA EXAMPLES

function declaration with lambda

```
auto f = [] { return 42; };
std::cout << f() << std::endl;          // prints 42
```

```
[] (const std::string& aLHS, const std::string& aRHS) { return aLHS.size() < aRHS.size() };
```

```
[lSize] (const std::string& aString) { return aString.size() < lSize; };
```

capture lSize

LAMBDA CAPTURES (SELECTED)

[]	Lambda does not use variables from the enclosing environment. Variables from the environment cannot be accessed.
[identifier list]	The variables listed in the comma-separated identifier list are captured by value and copied into the body of lambda. The lambda sees only stored values. Updates in the environment do not affect lambda.
[&]	All variables in the environment are implicitly captured by reference. Updates in the environment affect lambda.
[=]	All variables in the environment are implicitly captured by value. Values are copied into the body of lambda. Updates in the environment do not affect lambda.
[this]	Capture current object by reference.
[*this]	Capture current object by value (copy).
[&, identifier list]	Implicit capture by reference of all variables in the environment, except those that occur in the identifier list. The identify list must not contain &. It can contain ***this**.
[=, reference list]	Implicit capture by value (copied into the body of lambda) of all variables in the environment, except those that occur in the identifier list. In the reference list all names must be preceded by &.

INDEXER WITH LAMBDA

```
template<typename T, size_t N = 20>
class IndexerByStringLambda : public BasicIndexer<T, N>
{
public:
    // Use call operator to access element
    T& operator()( const std::string& aKey,
                   auto AtoI = [] ( const std::string& aKey )
                   {
                       size_t lIndex = 0;

                       for ( size_t i = 0; i < aKey.size(); i++ )
                       {
                           lIndex = lIndex * 10 + (static_cast<size_t>(aKey[i]) - '0');
                       }

                       return lIndex;
                   })
    {
        // Call base class operator[] (performs range check)
        return BasicIndexer<T, N>::operator[](AtoI(aKey));
    }
};
```

Use call operator() as operator[] expects one argument only

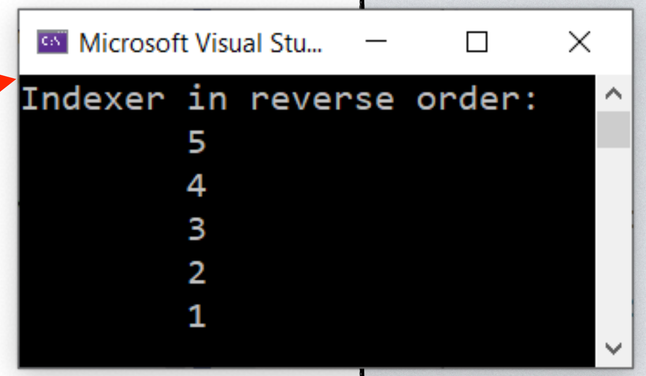
AUTO

```
auto f = [] (const string& aLHS, const string& aRHS) { return aLHS.size() < aRHS.size() };
```

- C++ has also introduced auto typing; we can declare variables using **auto** as type name.
- Using auto saves typing and prevents correctness and performance issues when dealing with complex types.
- Automatic type deduction via **auto** is no a free lunch. The programmer must guide the compiler to produce the right answer. Failing to do so can result in a wrong type altogether.
- Auto function parameters have been available since C++20.

INDEXER WITH BINARY KEYS

```
using IntIndexerByStringLambda = IndexerByStringLambda<int, 5>;

// Compile-time check of indexer properties using C++20 concepts and type traits.

IntIndexerByStringLambda IIndexerL;
std::string IBinaryKeys[] = { "000", "001", "010", "011", "100" };

auto BtoI = [] ( const std::string& aNumber )
{
    size_t IIndex = 0;

    for ( size_t i = 0; i < aNumber.size(); i++ ) { IIndex = (IIndex << 1) + (static_cast<size_t>(aNumber[i]) - '0'); }

    return IIndex;
};

for ( size_t i = 0; i < IIndexerL.size(); i++ ) { IIndexerL( IBinaryKeys[i], BtoI ) = static_cast<int>(i) + 1; }

std::cout << "Indexer in reverse order:" << std::endl;

for ( size_t i = IIndexerL.size(); i > 0; ) { std::cout << '\t' << IIndexerL( IBinaryKeys[--i], BtoI ) << std::endl; }
```

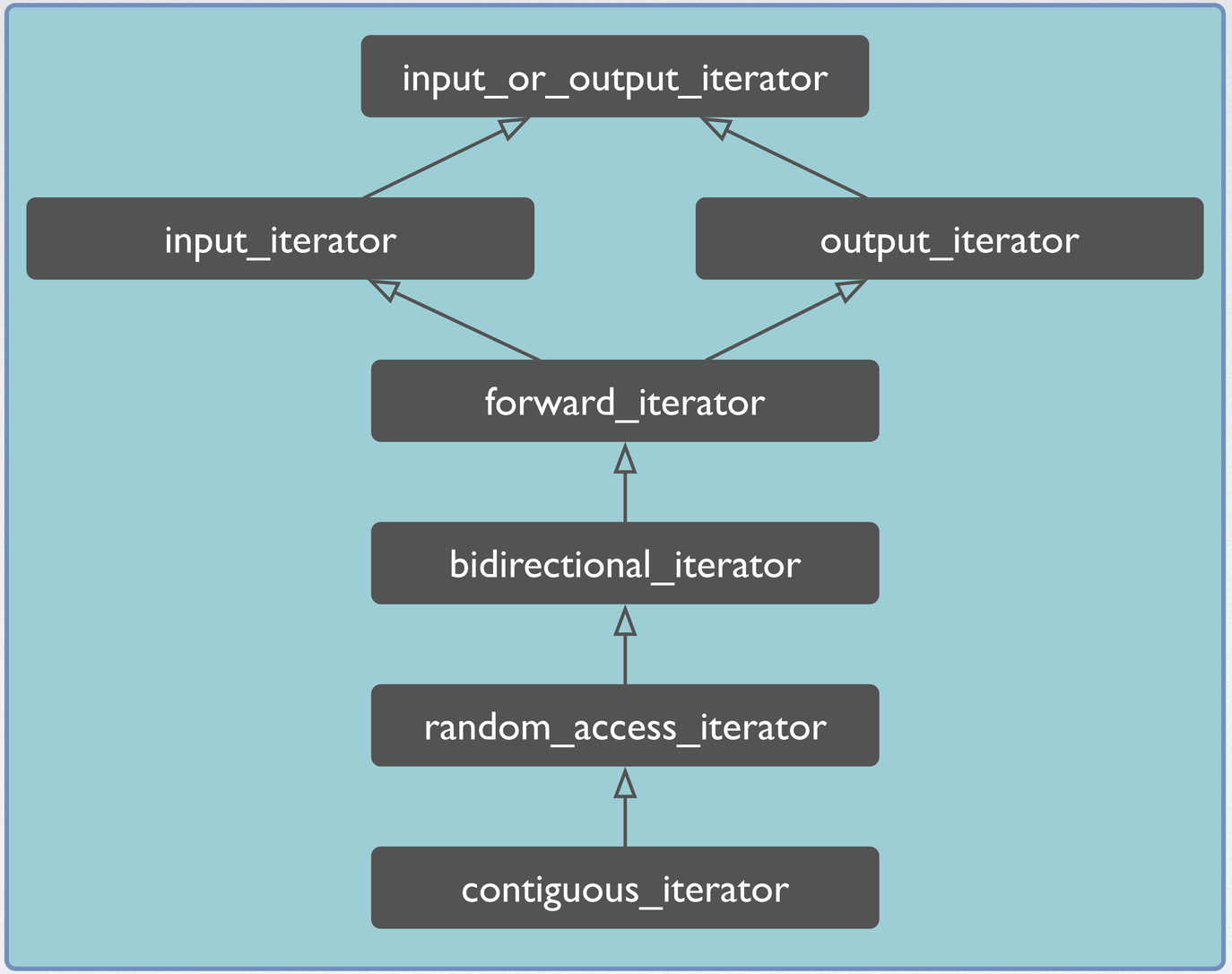


ITERATORS

SYSTEMATIC ITERATION OVER ELEMENTS: ITERATORS

- We can use a loop statement and a loop counter to visit all elements of an array in sequence.
- However, not all data types are arrays, and simple indexing may not suffice.
- Iterators offer programmers a suitable alternative to define iteration in a data type agnostic way.
- Conceptually, iterators are objects that implement the necessary infrastructure to iterate over a sequence. They do this via a common interface.
- The C++ ecosystem has popularized and uses five types of iterators. Iterator objects have the look and feel of pointers, and advancing an iterator means moving a pointer-like object forward or backward.

C++ ITERATOR CATEGORIES



ABILITIES OF ITERATORS

Category	Abilities
input_or_output_iterator	Objects of this type can be incremented and dereferenced
input_iterator	An input iterator allows its referenced value to be read, and both pre- and post-increment are supported.
output_iterator	An output iterator allows values to be written, and both pre- and post-increment are supported.
forward_iterator	A forward iterator is an input iterator supporting equality comparison and multi-pass (i.e., if iter1 == iter2, then ++iter1 == ++iter2).
bidirectional_iterator	A bidirectional iterator is a forward iterator supporting backward movement.
random_access_iterator	A random access iterator is a bidirectional iterator supporting advancement in constant time (via offsets) and indexing.
contiguous_iterator	A contiguous iterator is a random access iterator referring to elements that are contiguous in memory.

INPUT ITERATOR

Expression	Effect
*iter	provides read access to element at current iterator position
++iter	steps forward (returns new position; ITER& **operator++()**)
iter++	steps forward (returns old position; ITER **operator++(int)**)

INPUT ITERATOR EXAMPLE

```
class SimpleInputIterator
{
public:
    using difference_type = std::ptrdiff_t;     // to satisfy concept weakly_incrementable
    using value_type = int;                     // to satisfy concept indirectly_readable

    int& operator*() const;                     // const to satisfy concept indirectly_readable

    SimpleInputIterator& operator++();
    void operator++(int);
};

static_assert(std::input_iterator<SimpleInputIterator>);
```

WHY IS THE DEFERENCE OPERATOR **CONST**?

- Types that are indirectly readable via applying **operator*** include pointers, smart pointers, and iterators.

- In C++, the **const**-ness of such types does not affect whether it can be dereferenced. For example, if we dereference an **int*** or an **int*** **const** (a constant pointer to an **int**), the result is the same **int**&, a reference to an **int**. The top-level **const** does not matter. Iterators must follow this principle.

- The dereference operator **int**& **operator***() **const** returns a reference to an **int**. The iterator is constant, and **this** is constant within the scope of the dereference operator. In the context, the iterator is a constant pointer to an underlying datum for which a reference is returned (i.e., **int**&).

OUTPUT ITERATOR

Expression	Effect
*iter = value	provides write access to element at current iterator position
++iter	steps forward (returns new position; ITER& **operator++()**)
iter++	steps forward (returns old position; ITER **operator++(int)**)

An output iterator is like a "black hole".

FORWARD ITERATOR

Expression	Effect
*iter	provides read access to element at current iterator position
++iter	steps forward (returns new position; ITER& **operator++()**)
iter++	steps forward (returns old position; ITER **operator++(int)**)
iter1 == iter2	iterators iter1 and iter2 positioned at same element (allows multi-pass)

CAPTURING EQUALITY

- In C++, programmers can overload **operator==** to define equality for user-defined data types. Overloading **operator==** is similar to providing the method equals() in Java or Equals() in C#.
- The **operator==** is a binary operator that must return **bool**.
- Since C++ 20, programmers only have to provide an implementation for **operator==**. If type T defines **operator==**, then for some values i and j of type T, !(i == j) is equivalent to i != j.
- Overloading **operator!=** is no longer necessary. This makes defining equivalence consistent.
- **Note**: Some algorithms in C++ 17 and earlier rely on an explicitly defined **operator!=** (e.g., range loop).

FORWARD ITERATOR EXAMPLE

```
class SimpleForwardIterator
{
public:
    using difference_type = std::ptrdiff_t;  // to satisfy concept weakly_incrementable
    using value_type = int;                  // to satisfy concept indirectly_readable

    int& operator*() const;                  // const to satisfy concept indirectly_readable

    SimpleForwardIterator& operator++();
    SimpleForwardIterator operator++(int);

    bool operator==(const SimpleForwardIterator&) const;
};

static_assert(std::forward_iterator<SimpleInputIterator>);
```

BIDIRECTIONAL ITERATOR

Expression	Effect
*iter	provides read access to element at current iterator position
++iter	steps forward (returns new position; ITER& **operator++()**)
iter++	steps forward (returns old position; ITER **operator++(int)**)
—iter	steps backward (returns new position; ITER& **operator—()**)
iter—	steps backward (returns old position; ITER **operator—(int)**)
iter1 == iter2	iterators iter1 and iter2 positioned at same element (allows multi-pass)

RANDOM ACCESS ITERATOR

- Refinement of bidirectional iterator

Expression	Effect
iter[n]	provides read access to element at position n
iter += n	steps n elements forward (or backward)
iter -= n	steps n elements backward (or forward)
n + iter	returns iterator positioned n elements forward
iter + n	returns iterator positioned n elements forward
n - iter	returns iterator position n elements backward
iter1 - iter2	returns distance between iter1 and iter2

A BASIC ITERATOR EXAMPLE

```
template<typename T, size_t N>
class BasicIterator
{
private:
   T* fElements;                                          // underlying collection (array type decays to pointer)
    size_t fPosition;                                     // iterator position

public:
    using iterator = BasicIterator<T,N>;                  // iterator type alias
    using difference_type = std::ptrdiff_t;               // to satisfy concept weakly_incrementable
    using value_type = T;                                 // to satisfy concept indirectly_readable

    BasicIterator( T aElements[N] = nullptr ) noexcept;   // array decays to pointer, default constructible

   T& operator*() const noexcept;                         // const to satisfy concept indirectly_readable

    iterator& operator++() noexcept;                      // prefix increment
    iterator operator++( int ) noexcept;                  // postfix increment

    bool operator==( const iterator& aOther ) const noexcept;   // iter1 == iter2

    iterator begin() const noexcept;                      // return fresh iterator positioned at start
    iterator end() const noexcept;                        // return fresh iterator position after last
};

static_assert(BasicForwardIterator<BasicIterator<int, 5>>);
```

USER-DEFINED CONCEPT BASICFORWARDITERATOR

```
template<typename T>
concept BasicForwardIterator =
std::forward_iterator<T> &&                    // a basic forward iterator is a forward iterator
requires( const T i, const  T j )
{     // { expression } -> type_constraint;
    { i.begin() } -> std::same_as<T>;          // basic forward iterator provides begin()
    { i.end() } -> std::same_as<T>;            // basic forward iterator provides end()
};
```

- BasicForwardIterator is a user-defined concept that models the algorithmic guarantee that a basic forward iterator can be safely used in C++ range loops.

- The C++ standard requires that an iterator expression must provide definitions for the methods begin() and end() within the scope of the iterator expression type. The C++ concept BasicForwardIterator explicitly captures this requirement and allows the compiler to issue more meaningful error messages if either method is missing.

- Defining concepts is generally beyond the scope of COS30008.

ITERATOR CONSTRUCTOR

BasicIterator is trivially copyable

```
BasicIterator( T aElements[N] ) noexcept :
    fElements(aElements),
    fPosition(0)
{}
```

initialize member variables

DEREFERENCE OPERATOR - ELEMENT ACCESS

```
T& operator*() const noexcept
{
    return const_cast<T&>(fElements[fPosition]);
}
```

- The deference operator returns a read-write reference to the element at the current iterator position.
- No range check is required. The **operator*** () should only be called if the iterator has not yet reached the end of the underlying collection.
- The cast const_cast<T&> removes the **const**-ness of the iterator (see above).

PREFIX INCREMENT

- The prefix increment operator advances the iterator and returns a reference to the current iterator object:

```
iterator& operator++() noexcept
{
    fPosition++;

    return *this;
}
```

return a reference to current object

POSTFIX INCREMENT

- The postfix increment operator advances the iterator and returns a copy of the old iterator:

```
Iterator operator++( int ) noexcept
{
    iterator old = *this;

    ++(*this);          // forward to prefix increment

    return old;
}
```

dummy argument; used by compiler

return a copy of old iterator (old position)

EQUIVALENCE

```
bool operator==( const iterator& aOther ) const noexcept
{
    return  fElements == aOther.fElements.() &&          // collection
            fPosition == aOther.fPosition;               // position
}
```

- Two iterators are equal if and only if they refer to the same element:
  - The underlying collection is the same.
  - The iterators are positioned on the same element.

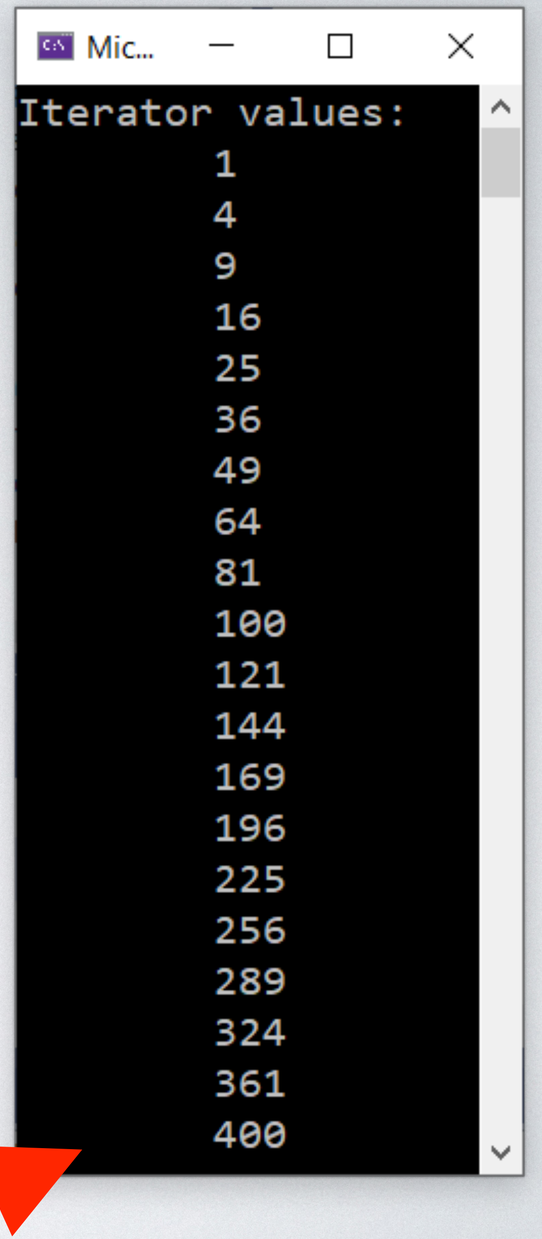
AUXILIARY METHODS

```
iterator begin() const noexcept
{
    iterator iter = *this;

    Result.fPosition = 0;

    return iter;
}

iterator end() const noexcept
{
    iterator iter = *this;

    Result.fPosition = N;

    return iter;
}
```

- The methods begin() and end() return fresh iterators positioned at the first and after the last element, respectively.
- The names and implementation of these auxiliary methods follow standard practices. The compiler may look for them when you use a range-loop.

ITERATOR APPLICATION

```
constexpr size_t ISize = 20;
int IArray[ISize];
BasicIterator<int, ISize> Iter1( IArray );
int i = 1;

for ( ; Iter1 != Iter1.end(); Iter1++ )
{
    *Iter1 = i * i;
    i++;
}

std::cout << "Iterator values:" << std::endl;

for ( BasicIterator Iter2 = Iter1.begin(); Iter2 != Iter2.end(); Iter2++ )
{
    std::cout << '\t' << *Iter2 << std::endl;
}
```



C++ RANGE-LOOP

- The traditional for statement in C++ reads:

for (init-statement; condition; expression) statement

- This form uses explicit loop variables, conditions, and increments over loop variables.

- C++11 introduces a simpler form, called range for statement, to iterate through the elements of a container or other sequence:

for (declaration : expression) statement

- This form is called range-loop, and the element expression denotes a sequence used to define a range-loop variable in declaration advanced in each iteration.

UNDERSTANDING THE RANGE LOOP

for (declaration : expression) statement

```
auto&& __range = expression;                  // C++ forwarding (move)
for ( auto __begin = begin-expression,        // begin() of expression object
           __end = end-expression;            // end() of expression object
      __begin != __end;                       // range-loop condition
      ++__begin )                             // advance iterator
{
  declaration = *__begin;                     // set loop variable to current element
  statement;                                  // range-loop body
}
```

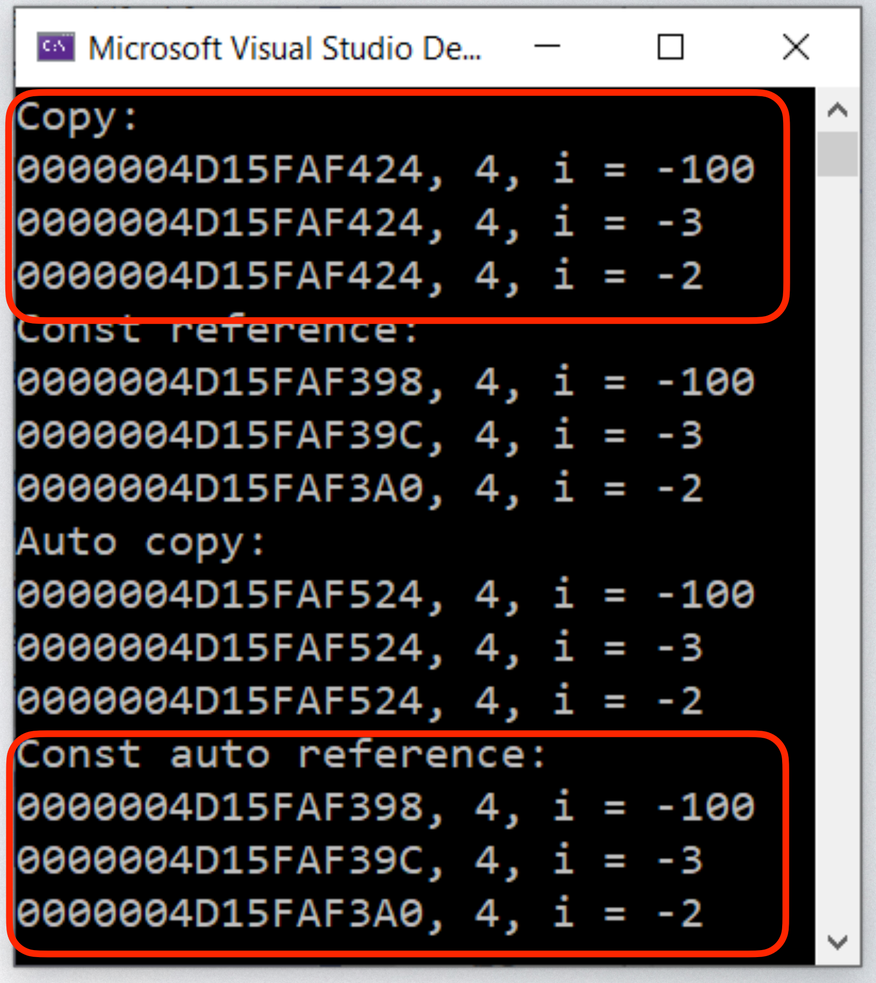
USING THE RANGE-LOOP

```
int lArray[] = {-100,-3,-2};

std::cout << "Copy: " << std::endl;

for ( int i : lArray )
{
	std::cout << std::hex << &i << "," << sizeof(i) << std::endl;
}

std::cout << "Const reference: " << std::endl;

for ( const int& i : lArray )
{
	std::cout << std::hex << &i << "," << sizeof(i) << std::endl;
}

std::cout << "Auto copy: " << std::endl;

for ( auto i : lArray )
{
	std::cout << std::hex << &i << "," << sizeof(i) << std::endl;
}

std::cout << "Const auto reference: " << std::endl;

for ( const auto& i : lArray )
{
	std::cout << std::hex << &i << "," << sizeof(i) << std::endl;
}
```

- Case 1: read-write variable
- Case 2: constant reference
- Case 3: auto variable
- Case 4: constant auto reference

RANGE-LOOP BEHAVIOR

```
int IArray[] = {-100,-3,-2};

std::cout << "Copy: " << std::endl;

for ( int i : IArray )
{
	std::cout << std::hex << &i << ", " << std::dec << sizeof(i) << ", i = " << i << std::endl;
}

std::cout << "Const reference: " << std::endl;

for ( const int& i : IArray )
{
	std::cout << std::hex << &i << ", " << std::dec << sizeof(i) << ", i = " << i  << std::endl;
}

std::cout << "Auto copy: " << std::endl;

for ( auto i : IArray )
{
	std::cout << std::hex << &i << ", " << std::dec << sizeof(i) << ", i = " << i  << std::endl;
}

std::cout << "Const auto reference: " << std::endl;

for ( const auto& i : IArray )
{
	std::cout << std::hex << &i << ", " << std::dec << sizeof(i) << ", i = " << i  << std::endl;
}
```



WHICH RANGE-LOOP PATTERN TO USE

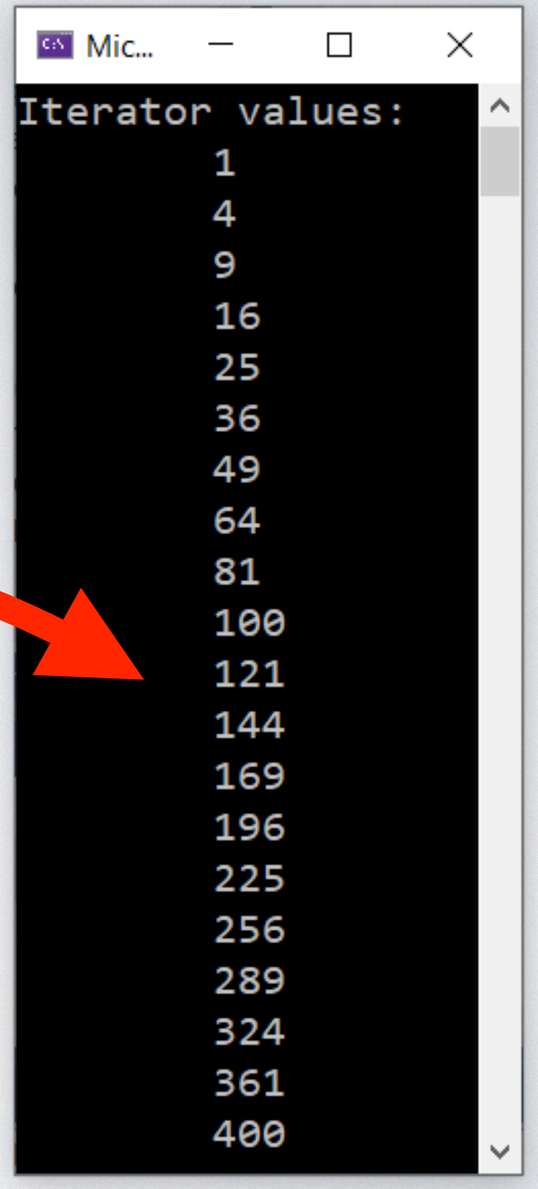
- In a range-loop, always use a reference variable. This avoids unnecessary copies.

- Prefer auto to explicit type declarations. Iterator types can be quite complex and hard to express. Using **auto**, automatic type deduction, simplifies things dramatically.

- You still need to understand what type deduction means and what it produces. Code becomes less readable as fewer explicit details are available.

ITERATOR APPLICATION WITH RANGE-LOOP

```
constexpr size_t ISize = 20;
int IArray[ISize];
BasicIterator<int, ISize> Iter1( IArray );
int i = 1;

// populate underlying collection (copy of Iter1)
for ( auto& item : Iter1 )
{
    item = i * i;
    i++;
}

std::cout << "Iterator values:" << std::endl;

// fetch elements from underlying collection (copy of Iter1)
for ( const auto& item : Iter1 )
{
    std::cout << '\t' << item << std::endl;
}
```



NOTE ON AUTO

- Auto saves typing and prevents correctness and performance issues when dealing with complex types.
- Automatic type deduction via **auto** is no free lunch. The programmer must guide the compiler to produce the right answer. Failing to do so can result in a wrong type altogether.
- The use of **auto** can hamper program comprehension. We may have to perform an in-depth study of the code base to understand what type we are dealing with and which methods can be safely invoked on a given object.
- Using **auto** can produce undefined behavior. The code still compiles fine, but the result may be unpredictable. In this case, an explicit type specification is required.

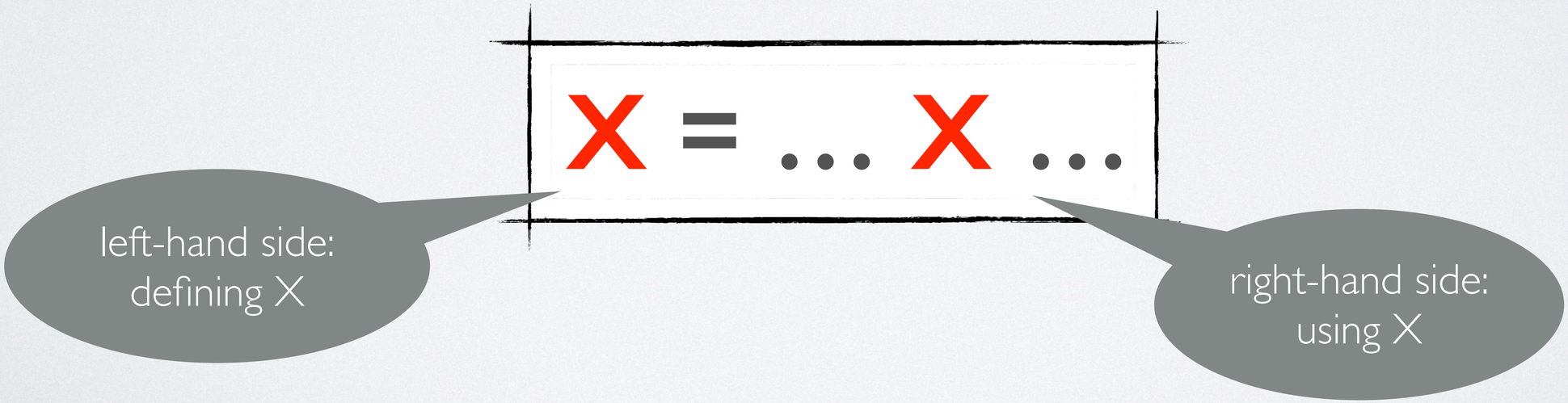
RECURSION

- If a procedure within its body calls itself, it is said to be recursively defined.
- This approach of program specification is called recursion and is found not only in programming.
- If we define a procedure recursively, at least one sub-problem must exist that can be solved directly, without calling the procedure again.
- A recursively defined procedure must always contain a directly solvable sub-problem. Otherwise, this procedure will not terminate.

PROBLEM-SOLVING WITH RECURSION

- Recursion is an important problem-solving technique in which a given problem is reduced to smaller instances of the same problem.

- The general structure of a recursive definition is



BASIC RECURSIVE PROBLEMS

FIBONACCI (PROGRAMMATIC REPRESENTATION)

- The Fibonacci numbers are the following sequence:

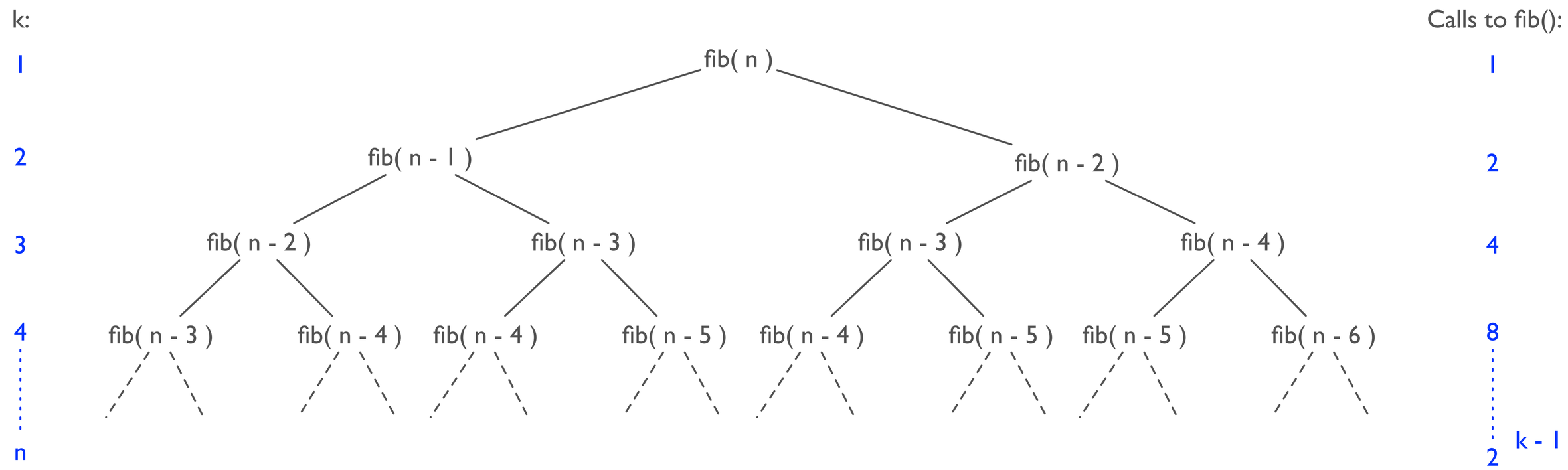
  0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, …

- In mathematical terms, the sequence F(n) of Fibonacci numbers is defined recursively as follows:

F(0) = 0
F(1) = 1
F(n) = F(n-i) + F(n-2)

```
size_t fib( size_t n ) noexcept
{
    if ( n <= 1 )
        return n;
    else
        return fib( n - 1 ) + fib( n - 2 );
}
```

UNFOLDING RECURSIVE FIBONACCI



FACTORIAL

- The factorial for positive integers is

$$n! = n * (n - 1) * \ldots * 1$$

- The recursive definition:

$$n! = \begin{cases} 1 & \text{if } n = 0 \\ n * (n-1)! & \text{if } n > 0 \end{cases}$$

FACTORIAL CALCULATION

- The recursive definition tells us exactly how to calculate a factorial:

```
4! = 4 * 3!                          Recursive step: n=4
   = 4 * (3 * 2!)                    Recursive step: n=3
   = 4 * (3 * (2 * 1!))              Recursive step: n=2
   = 4 * (3 * (2 * (1 * 0!)))        Recursive step: n=1
   = 4 * (3 * (2 * (1 * 1)))         Stop condition: n=0
   = 4 * (3 * (2 * 1))
   = 4 * (3 * 2)
   = 4 * 6
   = 24
```

FACTORIAL IMPLEMENTATION

```
size_t factorial( size_t n ) noexcept
{
    if ( n == 0 )
        return 1;
    else
        return n * factorial( n - 1 );
}

int main()
{
    std::cout << "6! = "  << factorial( 6 ) << std::endl;

    return 0;
}
```

TAIL RECURSION

- A function is called tail-recursive if it ends in a recursive call that does not build up any deferred operations.

```
long gcd( long x, long y ) noexcept
{
    if ( y == 0 )
        return x;
    else
        return gcd( y, x % y );
}
```

```
gcd( 1246, 234 ):
    gcd( 234, 76 )
    gcd( 76, 6 )
    gcd( 6, 4 )
    gcd( 4, 2 )
    gcd( 2, 0 )
    2
```

TAIL RECURSION TO LOOP

- A tail-recursive function can be converted into a loop.

```
long gcd( long x, long y ) noexcept
{
    if ( y == 0 )
        return x;
    else
        return gcd( y, x % y );
}
```

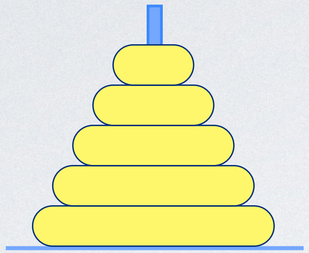
```
long gcdLoop( long x, long y ) noexcept
{
    while ( y != 0 )
    {
        long temp  = x;
        x = y;
        y = temp % y;
    }

    return x;
}
```

TOWERS OF HANOI

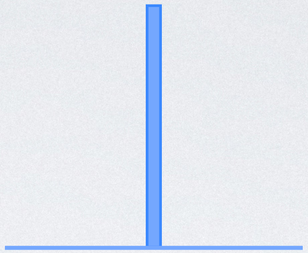
- **Problem**:
  - Move disks from a start peg to a target peg using the middle peg as temporary storage.
- **Challenge**:
  - All disks have a unique size, and at no time must a bigger disk be placed on top of a smaller one.

TOWERS OF HANOI: CONFIGURATION

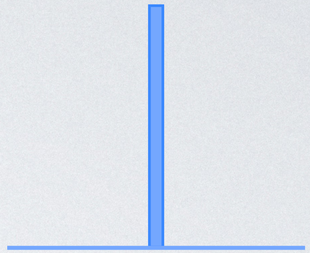
Start:



Start

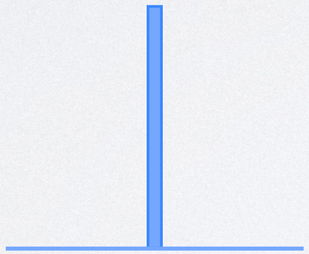


Middle

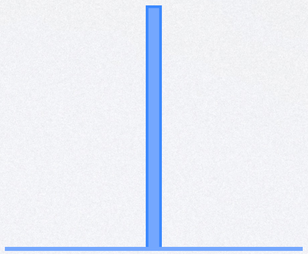


Target

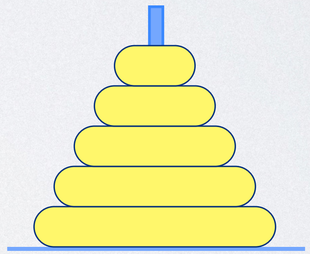
Finish:



Start



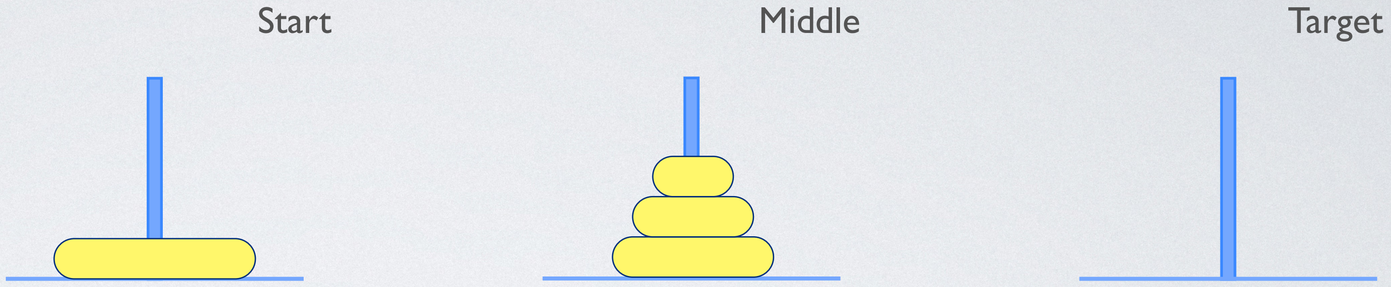
Middle



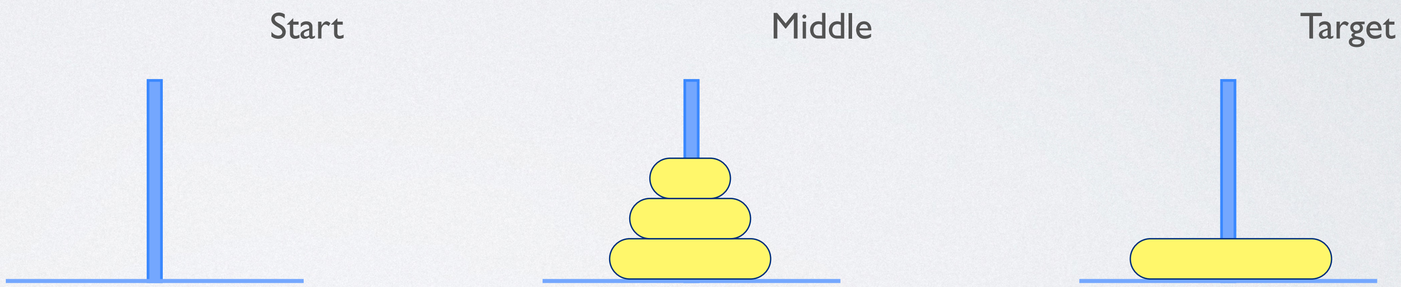
Target

RECURSIVE SOLUTION

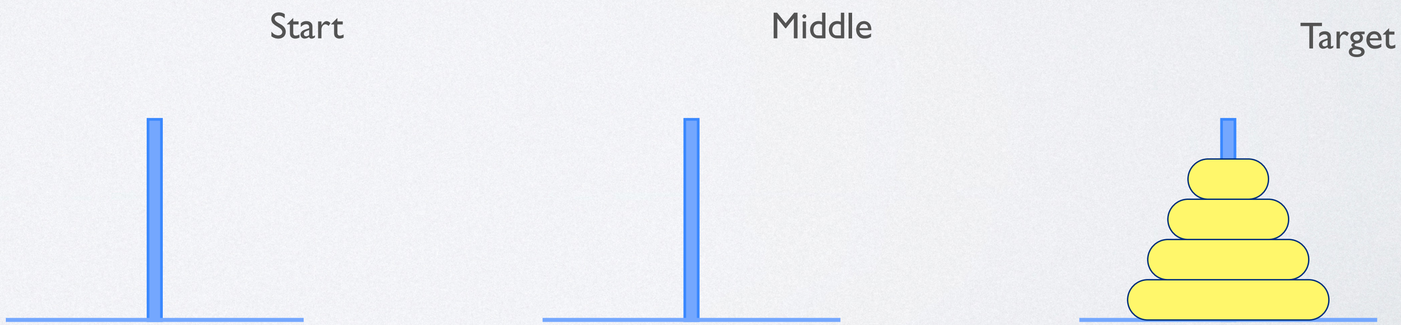
1. Move n-1 disks from Start to Middle:



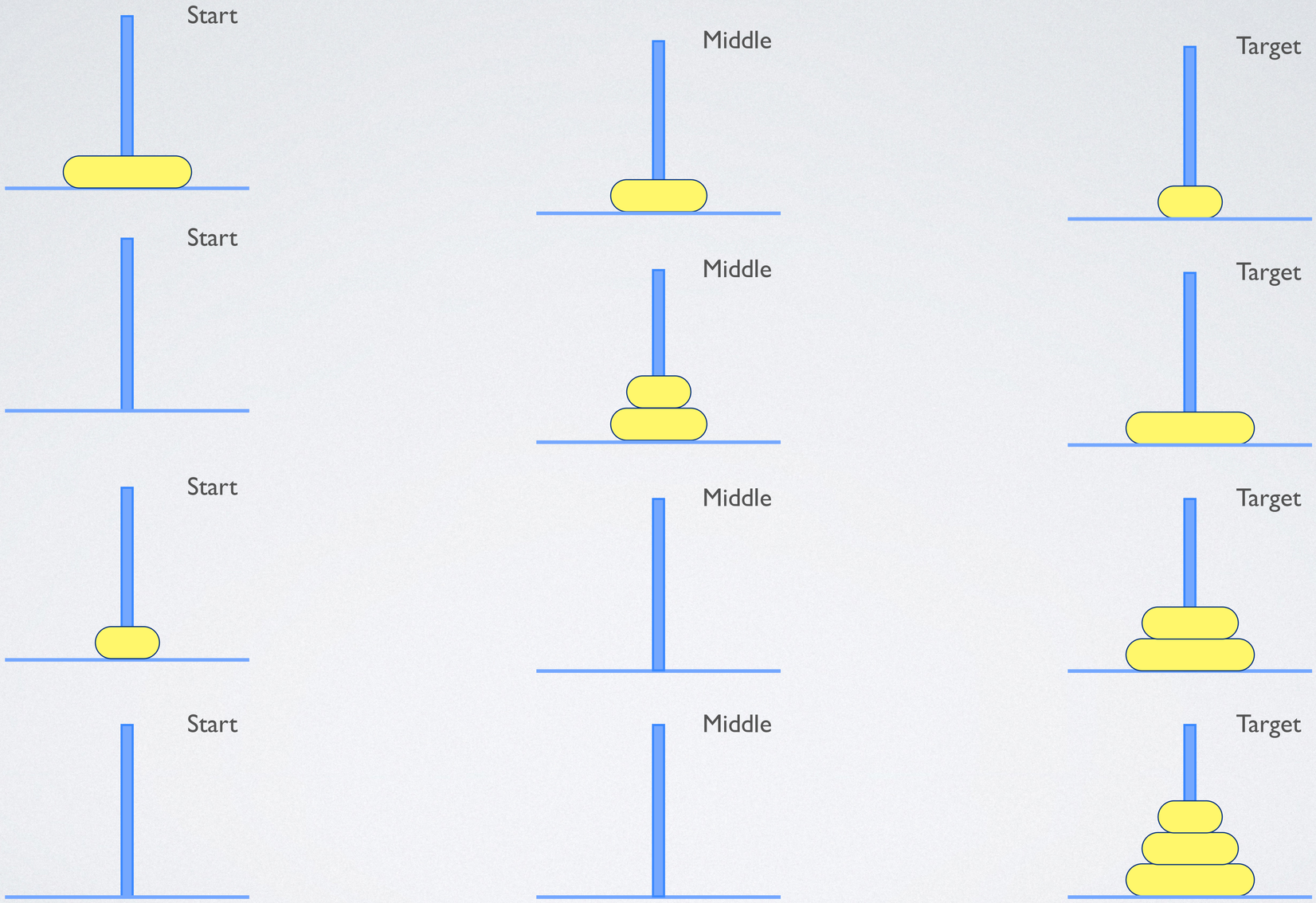
2. Move 1 disk from Start to Target:



3. Move n-1 disks from Middle to Target:

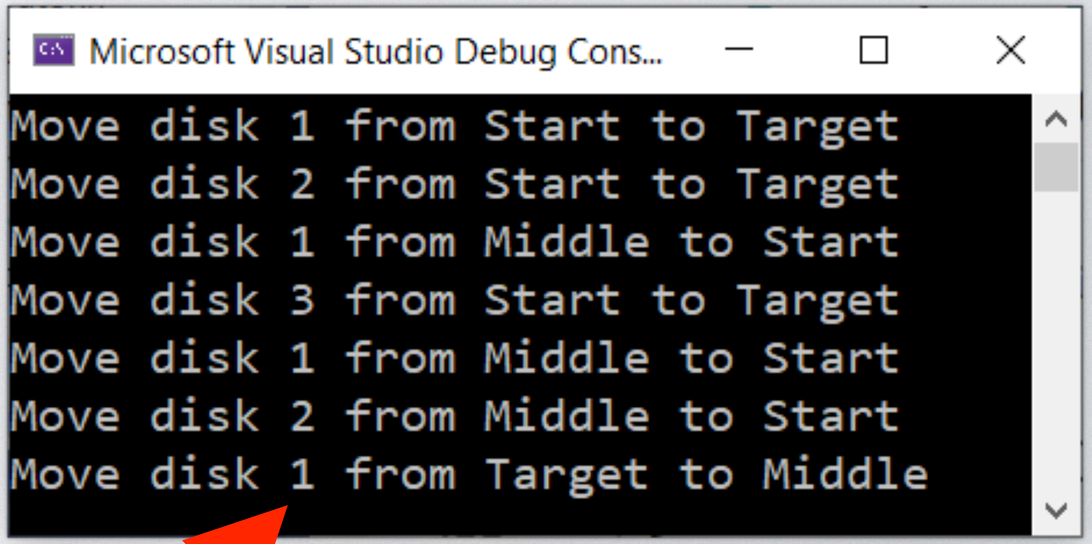


RECURSIVE SOLUTION: INTERMEDIATE



TOWERS OF HANOI: RECURSIVE PROCEDURE

```
void doTowersOfHanoi( size_t n,
                      const std::string& aStart,
                      const std::string& aMiddle,
                      const std::string& aTarget
                     ) noexcept
{
    if ( n > 0 )
    {
        doTowersOfHanoi( n - 1, aStart, aMiddle, aTarget );

        std::cout << "Move disk " << n << " from " << aStart
                  << " to " << aTarget << std::endl;

        doTowersOfHanoi( n - 1, aMiddle, aTarget, aStart );
    }
}

int main()
{
    doTowersOfHanoi( 3, "Start", "Middle", "Target" );

    return 0;
}
```



MERGE SORT

- Merge sort is a classical example of an algorithm based on the divide-and-conquer design strategy.

- Given a list L[low:high], let x be any index between low and high. Further, let A, B, and C denote the sublists A=L[low:x], B=L[x+1:high], and C=L[low:high], respectively. The problem of sorting C can be solved by sorting A (recursively), then sorting B (recursively), and finally calling the procedure Merge, which merges sorted lists A and B to obtain a sorted list C.

PROCEDURE MERGE SORT

```
procedure MergeSort( L[0:n-1], low, high )
begin
  if low < high then
    mid ← low + (high - low)/2        // prevent overflow bug
    MergeSort( L[0:n-1], low, mid )
    MergeSort( L[0:n-1], mid+1, high )
    Merge( L[0:n-1], low, mid, high )
  endif
end
```

PROCEDURE MERGE (DECREASING ORDER)

```
procedure Merge( L[0:n-1], low, mid, high )
begin
  for k ← low to mid do
    Temp[k] ← L[k]                          // copy left-to-right
  endFor
  for k ← mid to high do
    Temp[(high - k) + mid] ← L[k+1]         // copy right-to-left (prevent overflow)
  endFor
  CurPos1 ← low; CurPos2 ← high             // CurPos1 move right, CurPos2 move left
  for k ← low to high do
    if Temp[CurPos1] < Temp[CurPos2] then
      L[k] ← Temp[CurPos1]; CurPos1 ← CurPos1 + 1
    else
      L[k] ← Temp[CurPos2]; CurPos2 ← CurPos2 - 1
    endIf
  endFor
end
```

A POSSIBLE SOLUTION: MERGESORTER

```
template<typename T>
class MergeSorter
{
public:
    using array_type = std::vector<T>;

    MergeSorter( const array_type& aData ) noexcept;

    template<typename Op = std::greater<T>>
    void operator()( Op aTest = Op{}, std::ostream& aOStream = std::cout );

    const array_type& data() const noexcept;

private:
    using Comparator = std::function<bool(T,T)>;

    array_type fData;
    array_type fMergeData;
    Comparator fComparator;
    size_t indent;

    void doMergeSort( size_t aLow, size_t aHigh, std::ostream& aOStream ) noexcept;
    void doMerge( size_t aLow, size_t aMid, size_t aHigh ) noexcept;
};
```

parameterized over ordering relation

type of relational operator

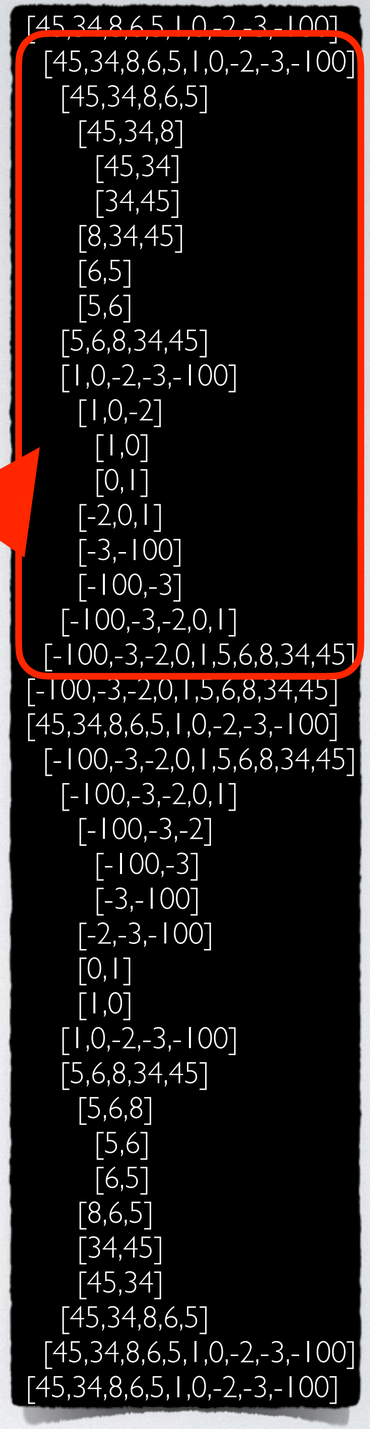
STD::FUNCTION

```
template<typename R, typename... Args>
class function<R(Args...)>
```

- Technically, a variable that stores a lambda is a function pointer. However, function pointers may lead to unreadable specifications or worse.
- Since C++11, C++ offers a function wrapper std::function for this purpose.
- Technically, std::function is a variadic template (it takes a variable number of arguments) that allows us to capture any function signature.

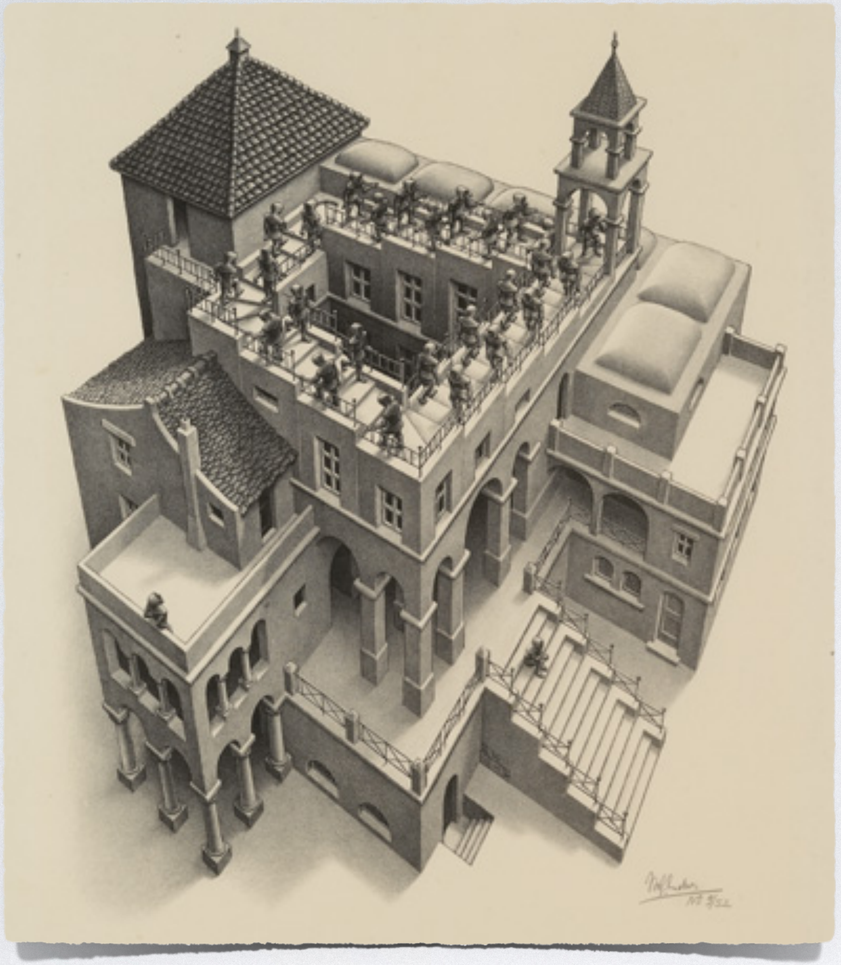
USING MERGE SORT

```
std::vector IData { 45, 34, 8, 6, 5, 1, 0, -2, -3, -100 };

MergeSorter ISorter( IData );

sendToStream( std::cout, IData ) << std::endl;

ISorter();

sendToStream( std::cout, ISorter.data() ) << std::endl;
sendToStream( std::cout, IData ) << std::endl;

ISorter( std::less{} );

sendToStream( std::cout, ISorter.data() ) << std::endl;
```

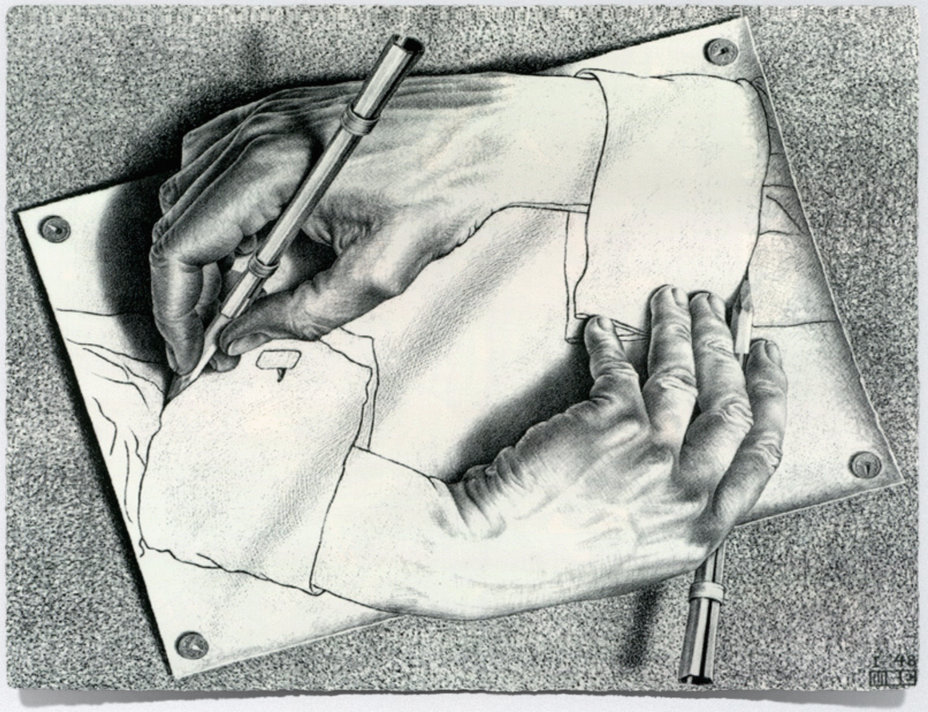


RECURSION IS A PREREQUISITE FOR LINKED LISTS

IMPOSSIBLE STRUCTURES



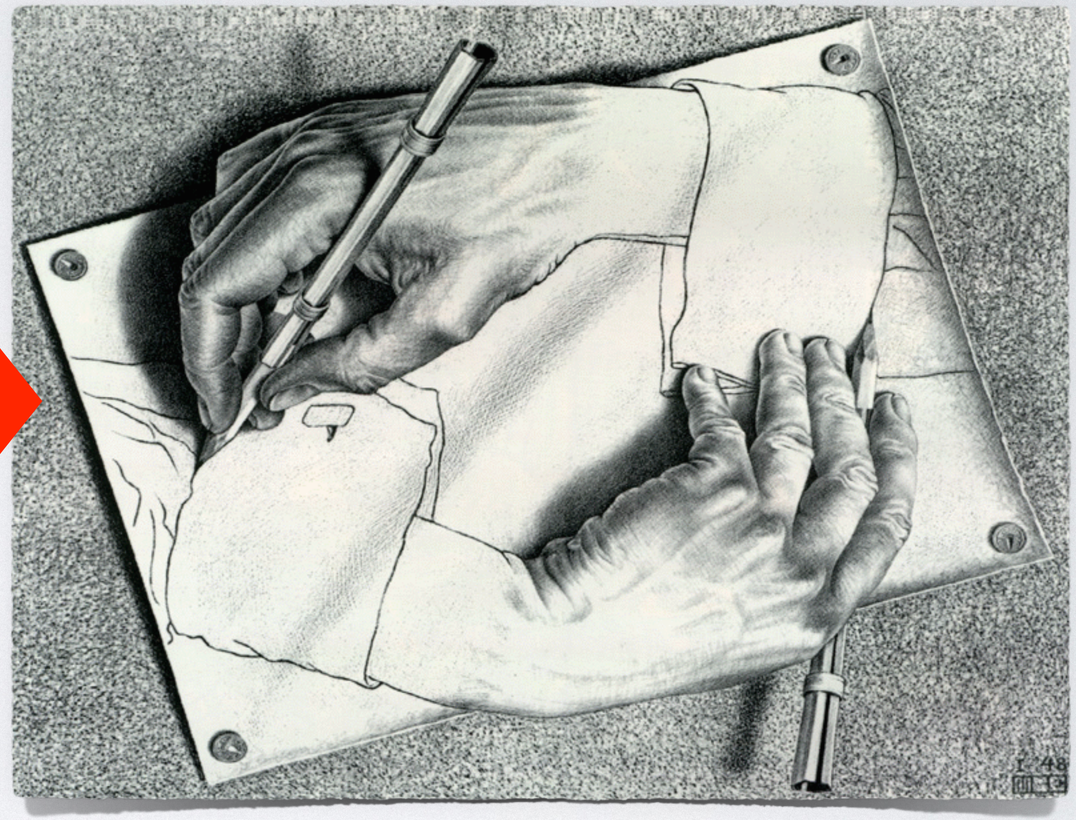
http://www.mcescher.com



IMPOSSIBLE STRUCTURES IN C++

```
#include "ClassB.h"

class ClassA
{
    use ClassB
};
```

```
#include "ClassA.h"

class ClassB
{
    use ClassA
};
```



FORWARD DECLARATION

- Unlike in Java or C#, mutual recursive classes in C++ are not automatically resolved.
- We must resolve the dependency manually and use forward declarations in specifications.
- In the implementations, we must include all specifications so that the compiler can resolve the dependencies.